

Compréhension des Réseaux de Neurones Artificiels (ANNs)



MonCoachData

 ANN

- Une grande partie de cette section sera consacrée à la théorie qui sous-entend nombre des idées que nous mettrons en œuvre avec le code.
- Faisons un rapide tour d'horizon de la manière dont nous allons progressivement comprendre les réseaux de neurones artificiels (Artificial Neural Networks - ANN)

- Sujets Théoriques
 - Du modèle Perceptron aux réseaux de neurones
 - Fonctions d'Activation
 - Fonctions de Coût d'erreur (Cost)
 - Réseaux de rétroaction (Feed Forward Networks)
 - Rétropropagation (BackPropagation)

- Sujets relatifs au codage
 - Syntaxe de TensorFlow 2.* Keras
 - ANN avec Keras
 - Régression
 - Classification
 - Exercices pour Keras ANN
 - Visualisations Tensorboard

C'est parti !

Modèle Perceptron



MonCoachData





Modèle Perceptron

- Pour commencer à comprendre le Deep Learning, nous allons construire nos modèles d'abstraction :
 - Neurone biologique seul
 - Perceptron
 - Modèle de Perceptron Multicouche
 - Réseau de Neurones Deep Learning





Modèle Perceptron

- Au fur et à mesure que nous apprendrons des modèles plus complexes, nous introduirons également des concepts tels que :
 - Fonctions d'Activation
 - Gradient Descent - Descente de Gradient
 - BackPropagation - rétropropagation





Modèle Perceptron

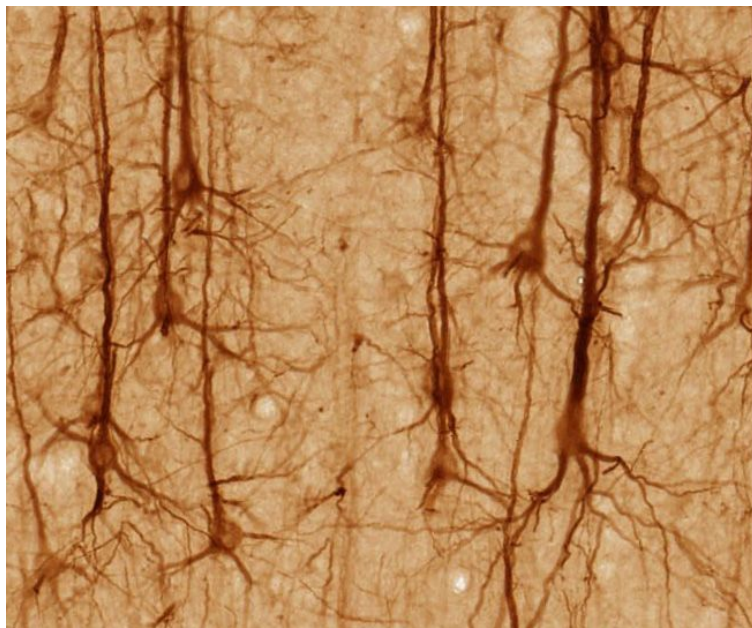
- Si l'idée générale du Deep Learning est de faire en sorte que les ordinateurs imitent artificiellement l'intelligence naturelle biologique, nous devrions probablement acquérir une compréhension générale du fonctionnement des neurones biologiques !





Modèle Perceptron

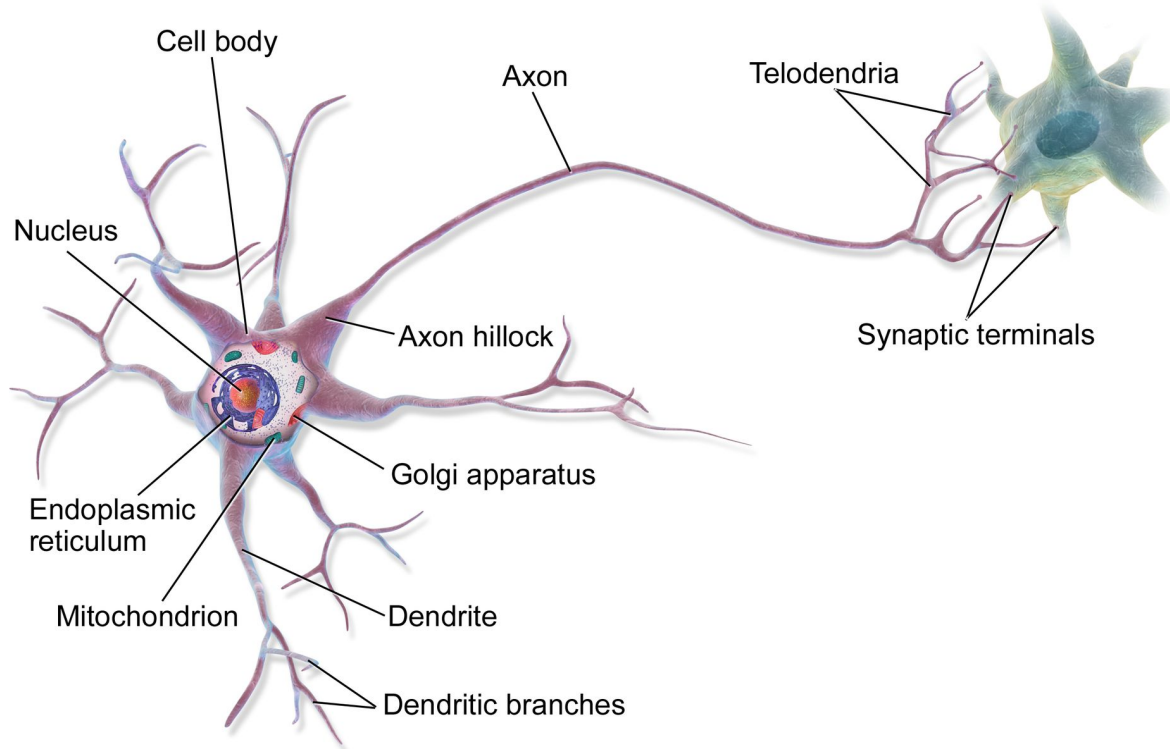
- Neurones colorés dans le cortex cérébral





Modèle Perceptron

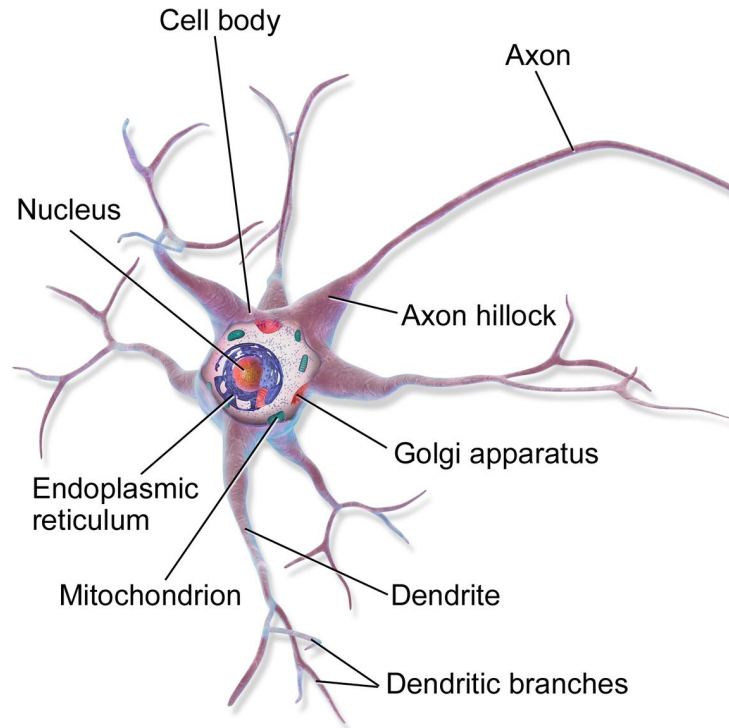
- Illustration des neurones biologiques





Modèle Perceptron

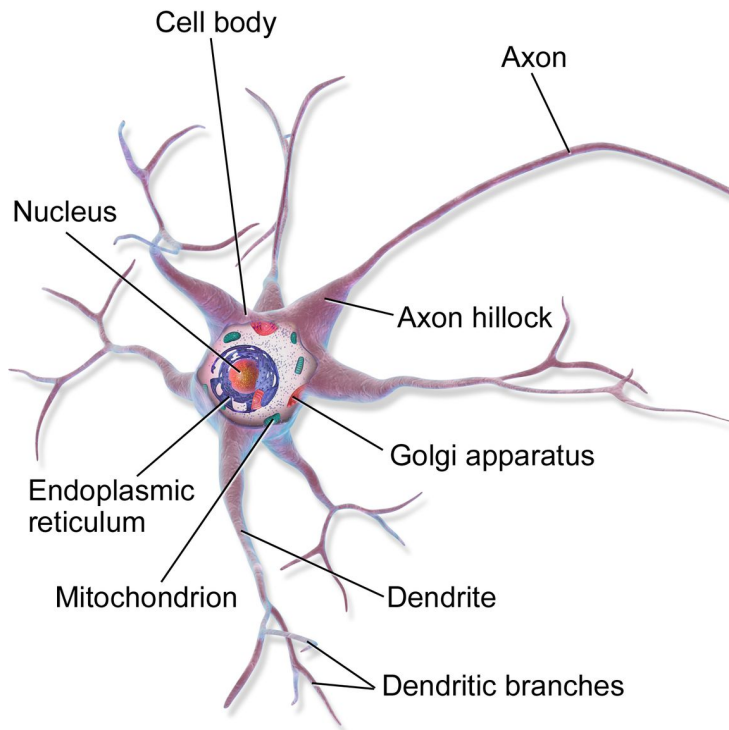
- Illustration des neurones biologiques





Modèle Perceptron

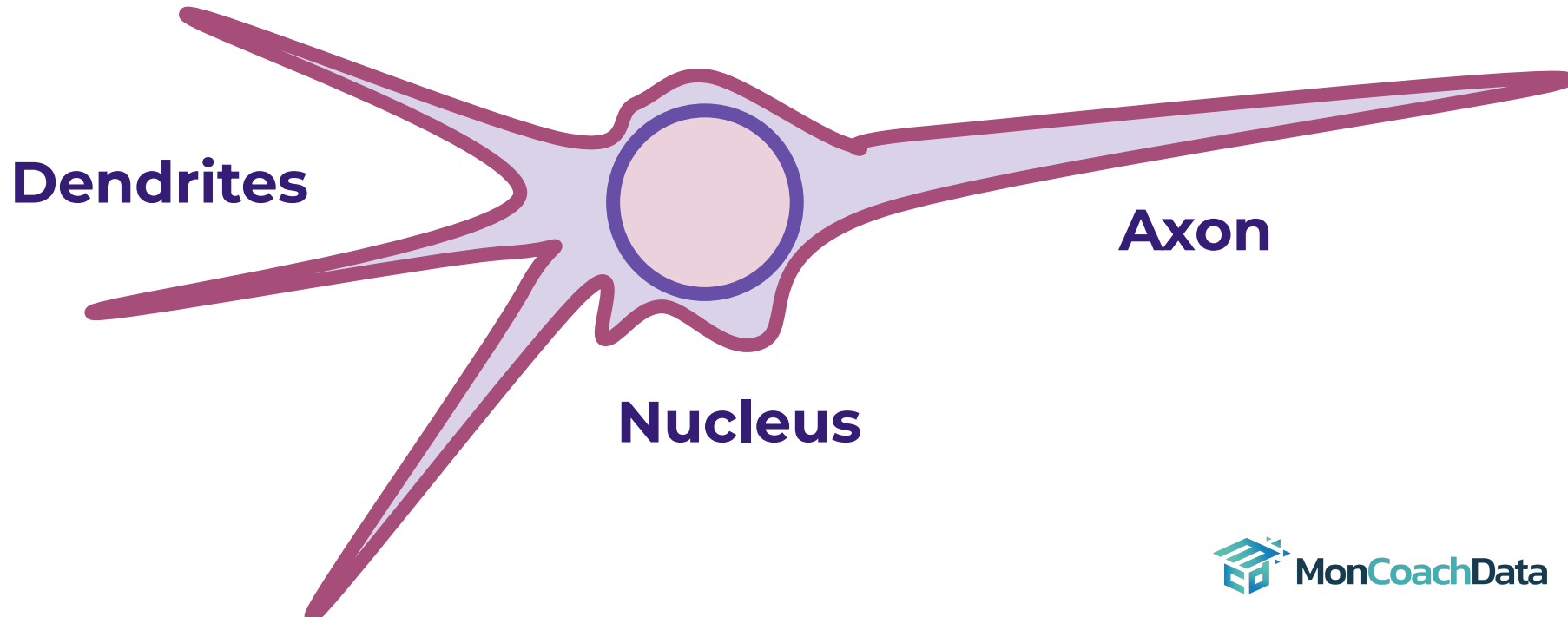
- Simplifions cela !





Modèle Perceptron

- Modèle simplifié de neurone biologique





Modèle Perceptron

- Un perceptron est une forme de réseau de neurones introduite en 1958 par Frank Rosenblatt.
- Étonnamment, même à l'époque, il voyait un énorme potentiel :
 - "...perceptron pourrait éventuellement être capable d'apprendre, de prendre des décisions et de traduire des langues."





Modèle Perceptron

- Cependant, en 1969, Marvin Minsky et Seymour Papert ont publié leur livre **Perceptrons**.
- Ce livre suggérait qu'il y avait de sérieuses limites à ce que les perceptrons pouvaient faire.
- Cela a marqué le début de ce que l'on appelle l'hiver de l'IA, avec peu de financement pour l'IA et les réseaux de neurones dans les années 1970.





Modèle Perceptron

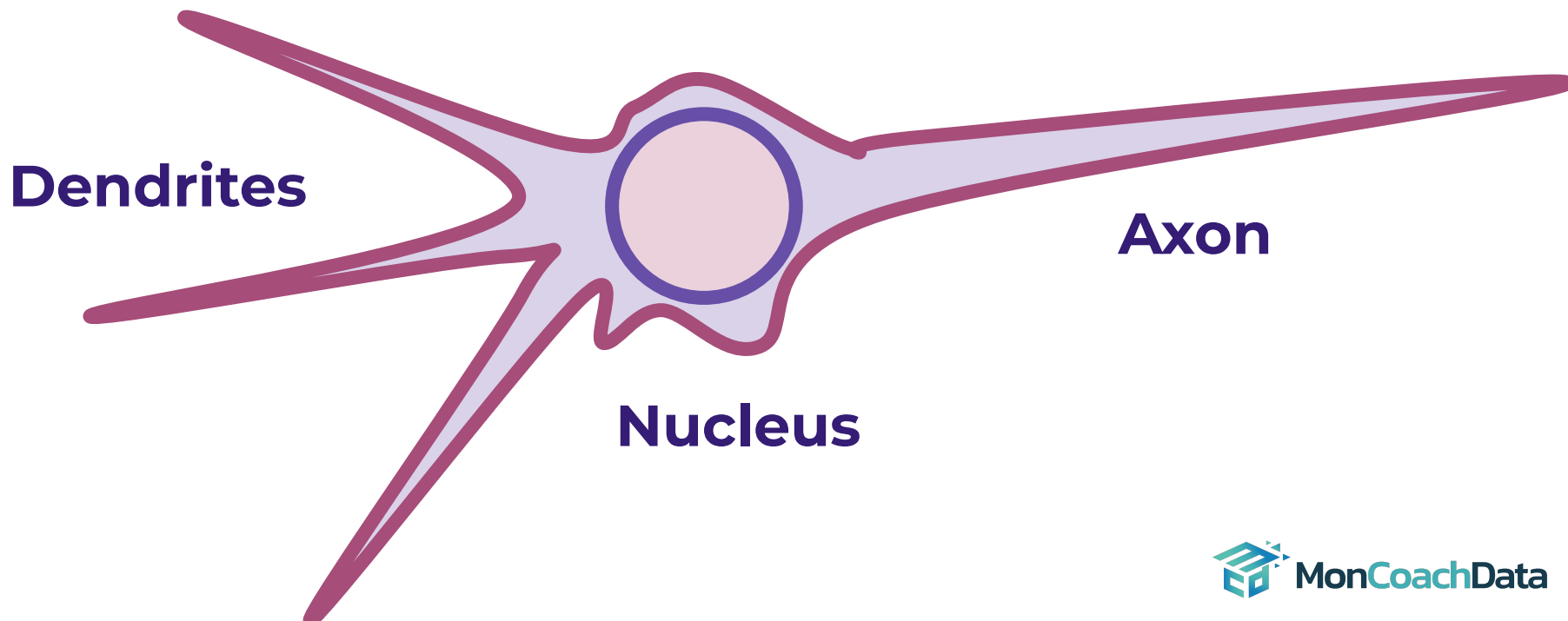
- Heureusement pour nous, nous connaissons maintenant l'incroyable puissance des réseaux de neurones, qui découlent tous du modèle simple du perceptron.





Modèle Perceptron

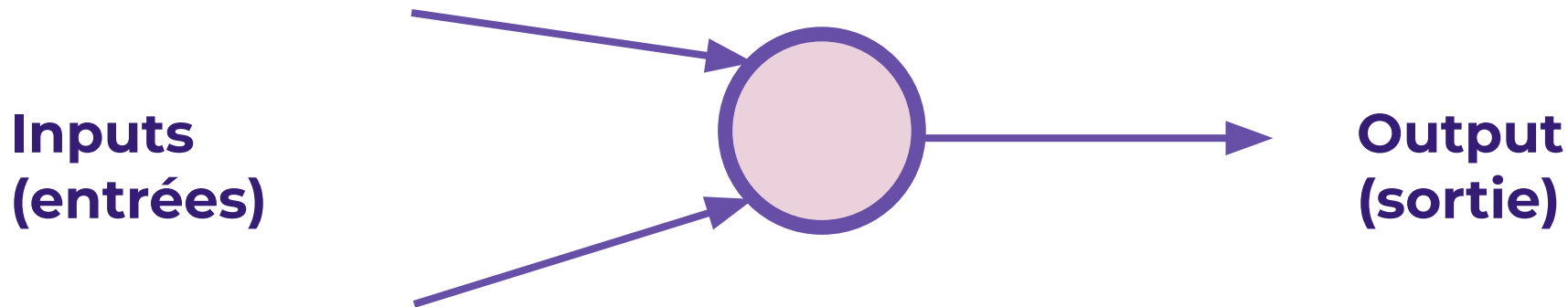
- Modèle Perceptron





Modèle Perceptron

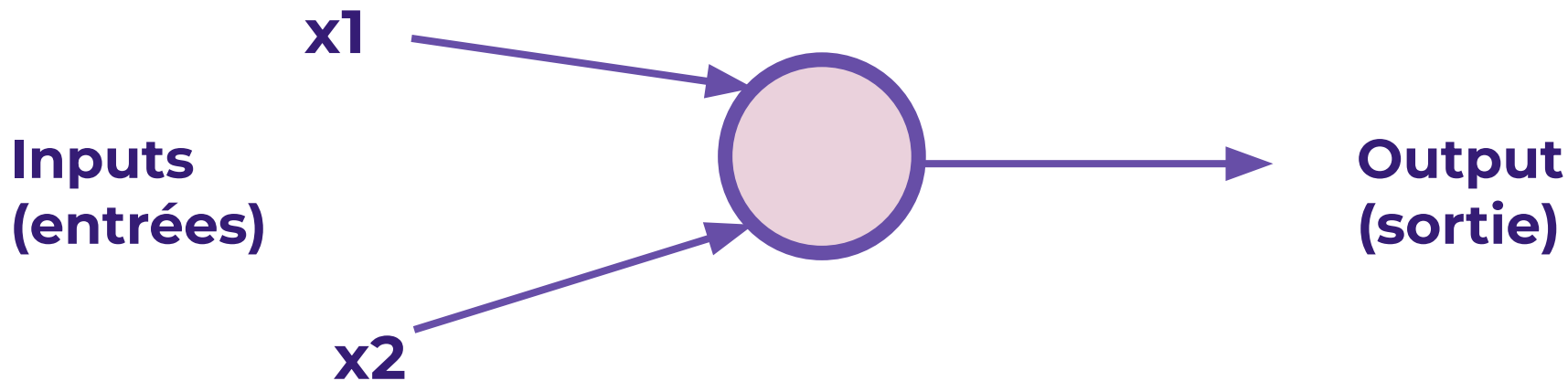
- Modèle Perceptron





Modèle Perceptron

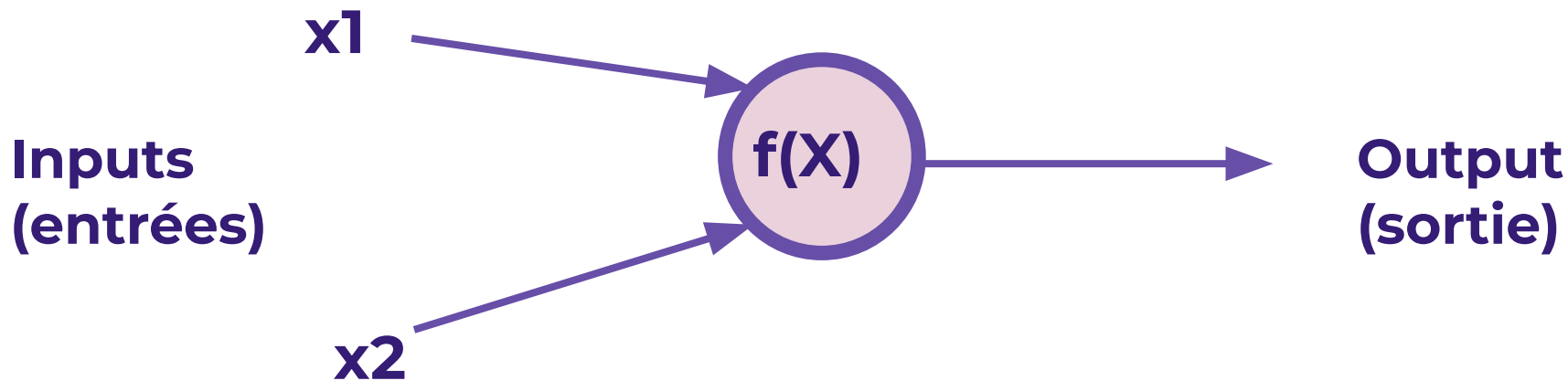
- Prenons un exemple simple





Modèle Perceptron

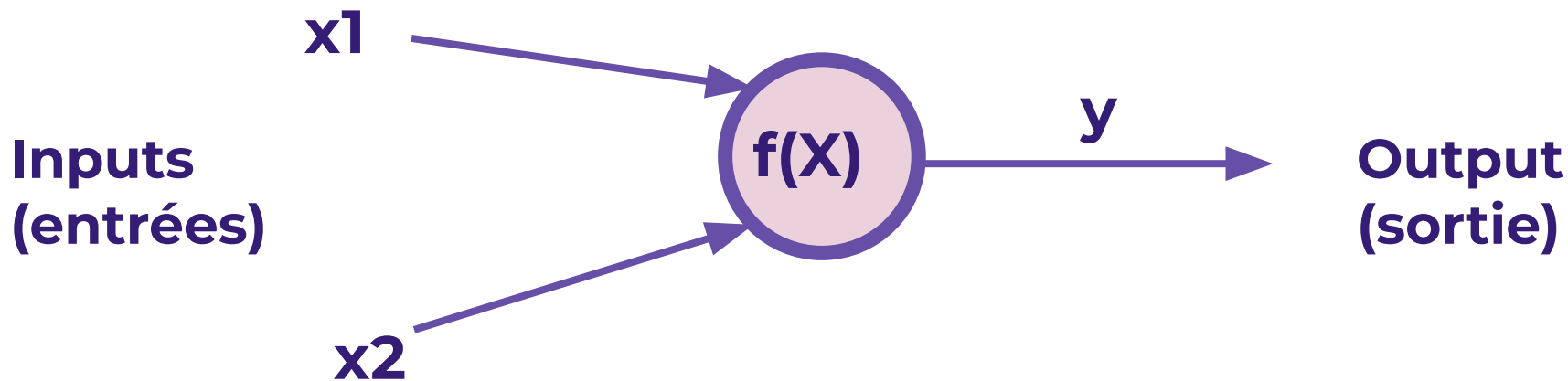
- Prenons un exemple simple





Modèle Perceptron

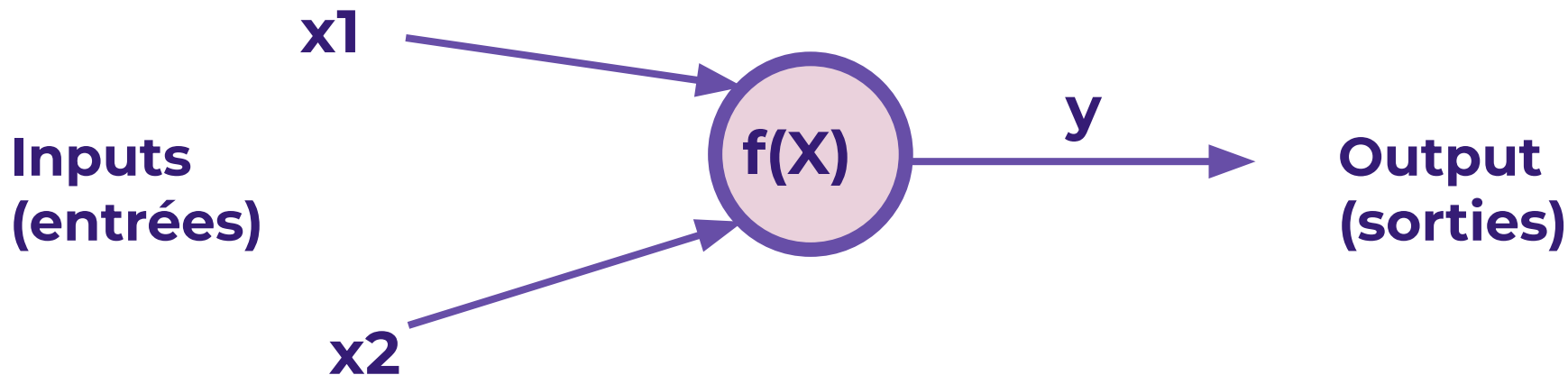
- Prenons un exemple simple





Modèle Perceptron

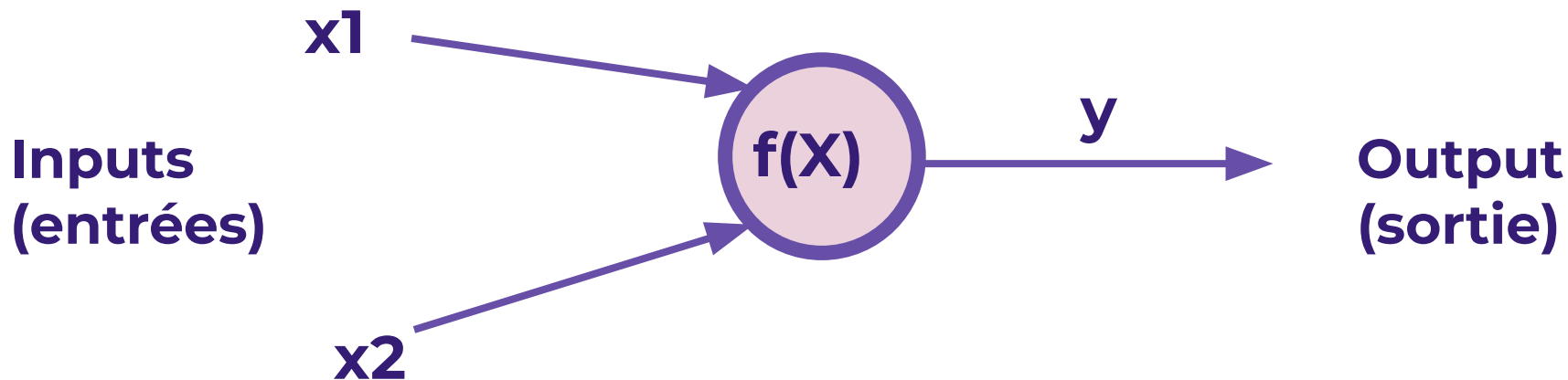
- Si $f(X)$ est juste une somme, alors $y = x_1 + x_2$





Modèle Perceptron

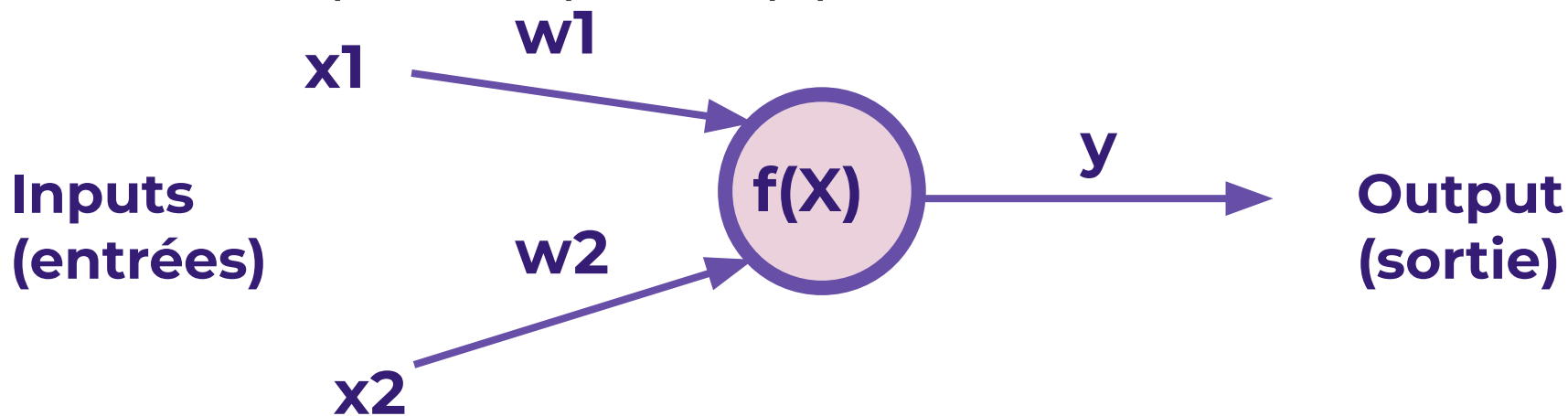
- De manière réaliste, nous voudrions pouvoir ajuster certains paramètres dans un but “d'apprentissage”





Modèle Perceptron

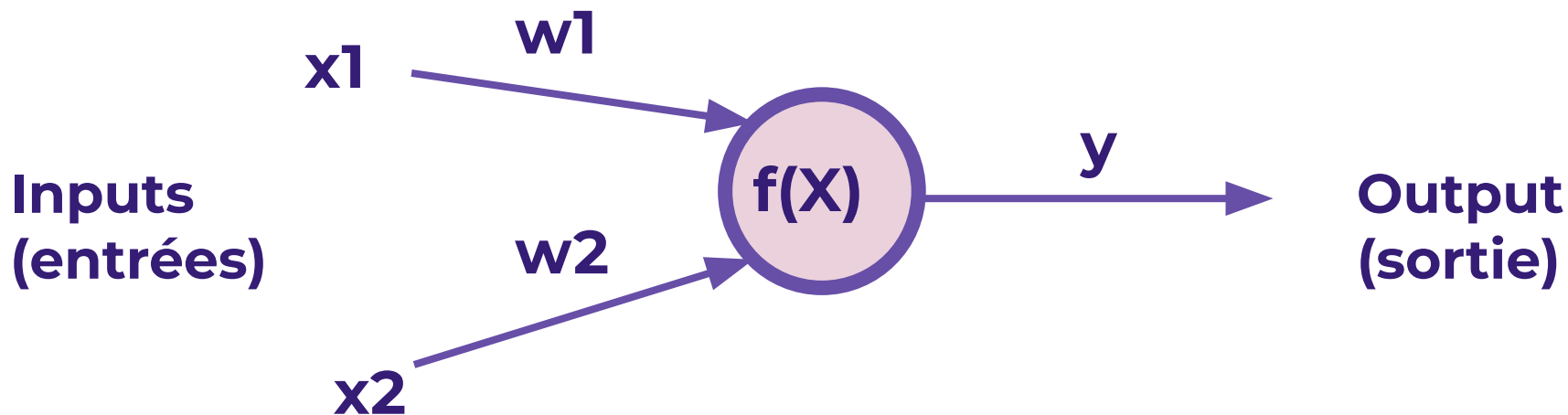
- Ajoutons un poids ajustable que nous multiplions par rapport à x





Modèle Perceptron

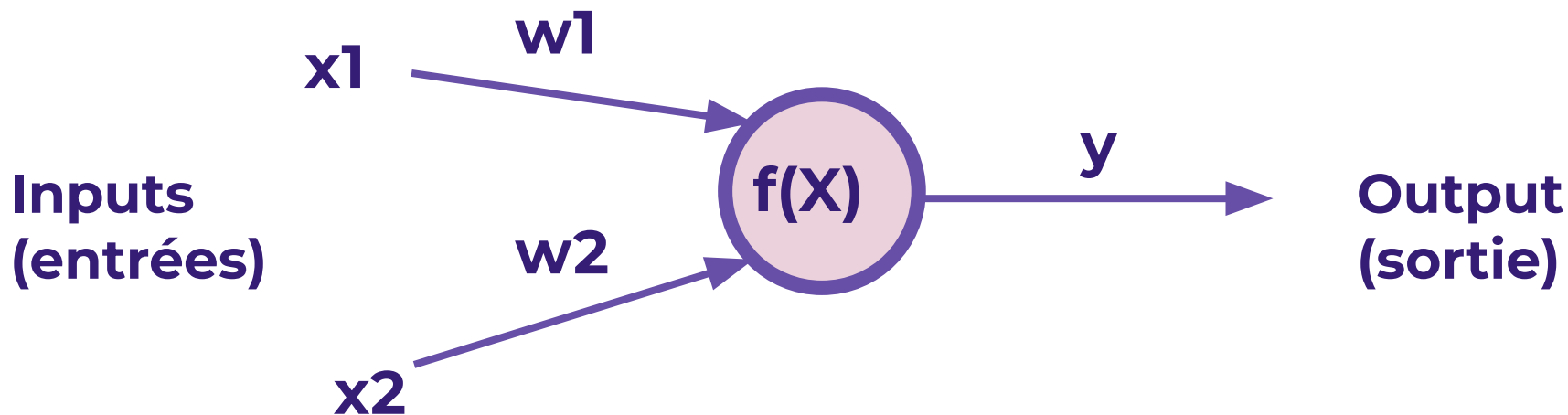
- Maintenant $y = x_1w_1 + x_2w_2$





Modèle Perceptron

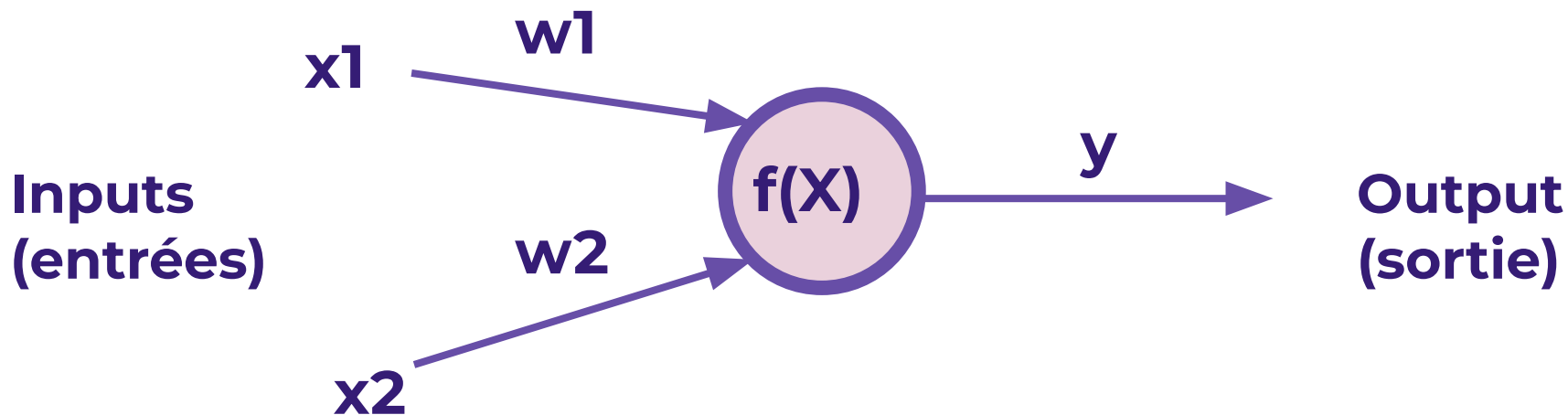
- Nous pourrions mettre à jour les **pondérations** pour affecter **y**





Modèle Perceptron

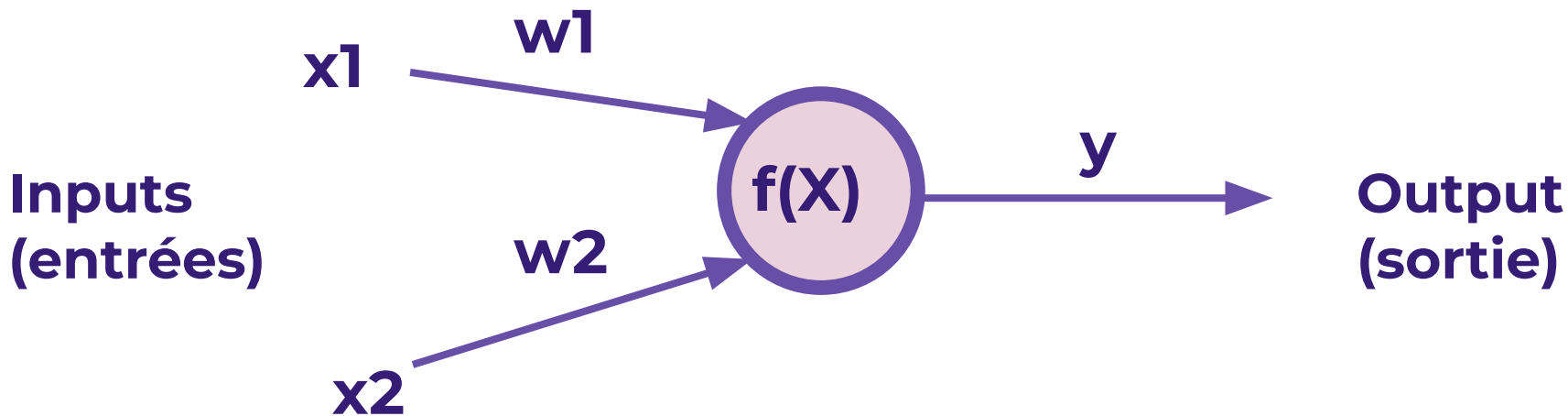
- Mais si un $\mathbf{x}$ est égal à zéro, $\mathbf{w}$ ne changera rien !





Modèle Perceptron

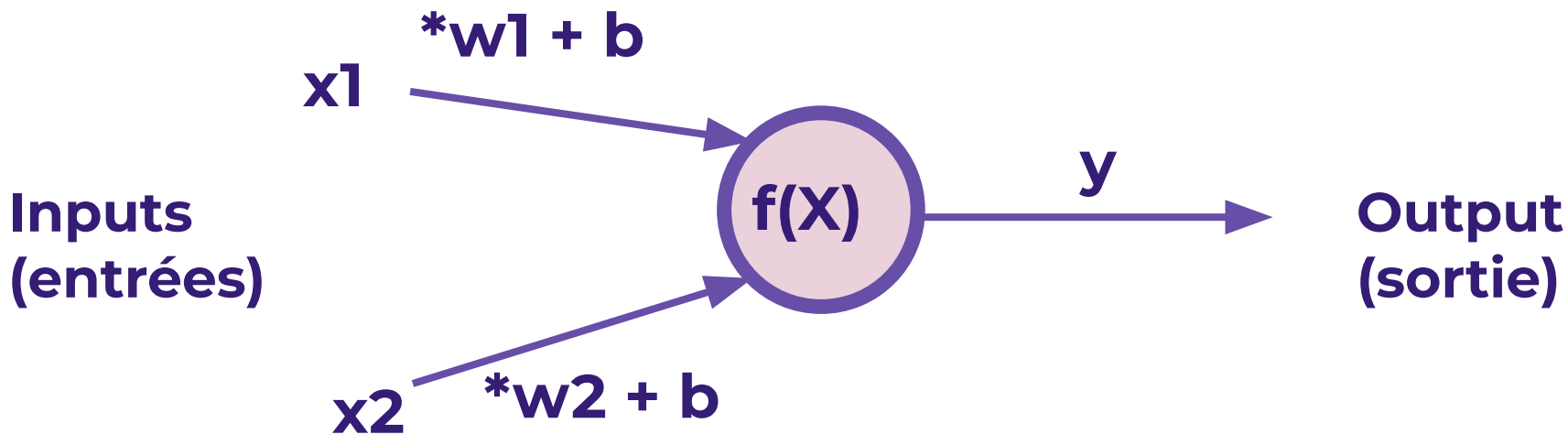
- Ajoutons un terme de **biais b** aux entrées.





Modèle Perceptron

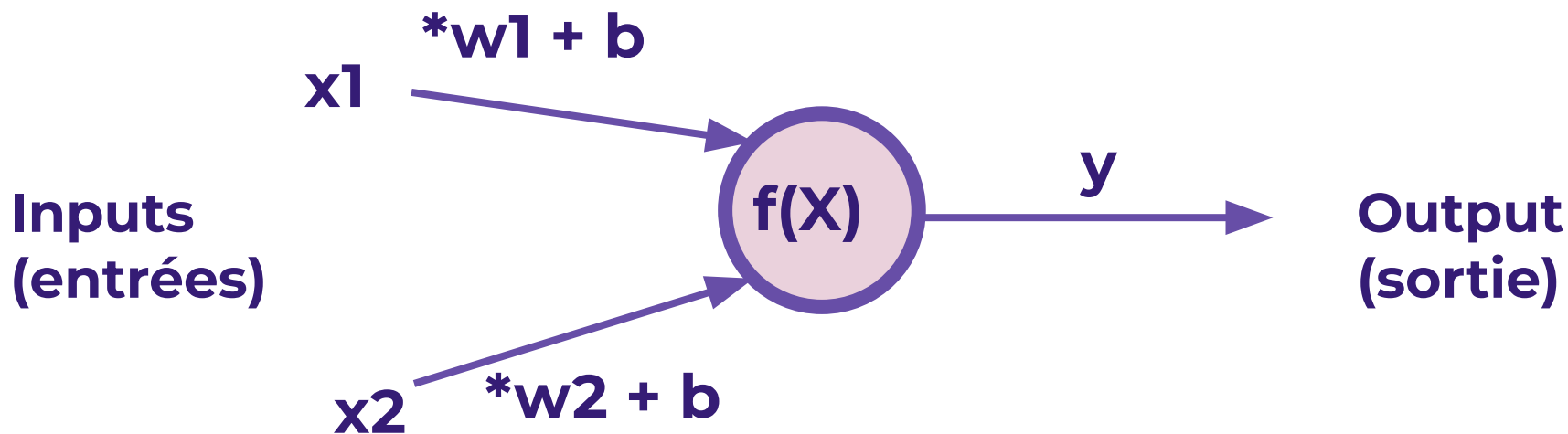
- Ajoutons un terme de **biais b** aux inputs.





Modèle Perceptron

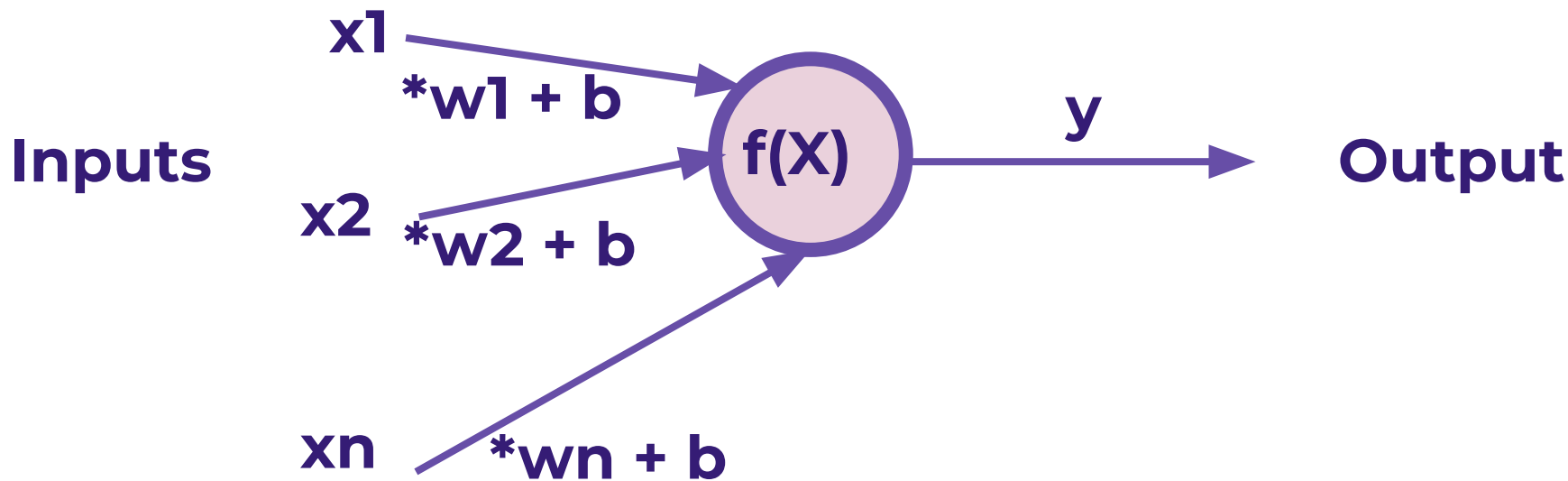
- $y = (x_1w_1 + b) + (x_2w_2 + b)$





Modèle Perceptron

- Nous pouvons étendre ce phénomène et le généraliser :





Modèle Perceptron

- Nous avons réussi à modéliser un neurone biologique comme un simple perceptron !
Mathématiquement, notre généralisation est :

$$\hat{y} = \sum_{i=1}^n x_i w_i + b_i$$





Modèle Perceptron

- Nous verrons plus tard comment nous pouvons étendre ce modèle pour que X soit un **tenseur** d'informations (une matrice à n dimensions).

$$\hat{y} = \sum_{i=1}^n x_i w_i + b_i$$





Modèle Perceptron

- Passons en revue ce que nous venons d'apprendre :
 - Nous comprenons les bases mêmes d'un neurone biologique
 - Nous avons vu comment nous pouvons créer un modèle de perceptron simple reproduisant les concepts fondamentaux d'un neurone.



Réseaux de Neurones





Réseaux de Neurones

- Un seul perceptron ne suffira pas pour apprendre des systèmes complexes.
- Heureusement, nous pouvons développer l'idée d'un perceptron unique, pour créer un modèle de perceptron multicouche (multi-layer).





Réseaux de Neurones

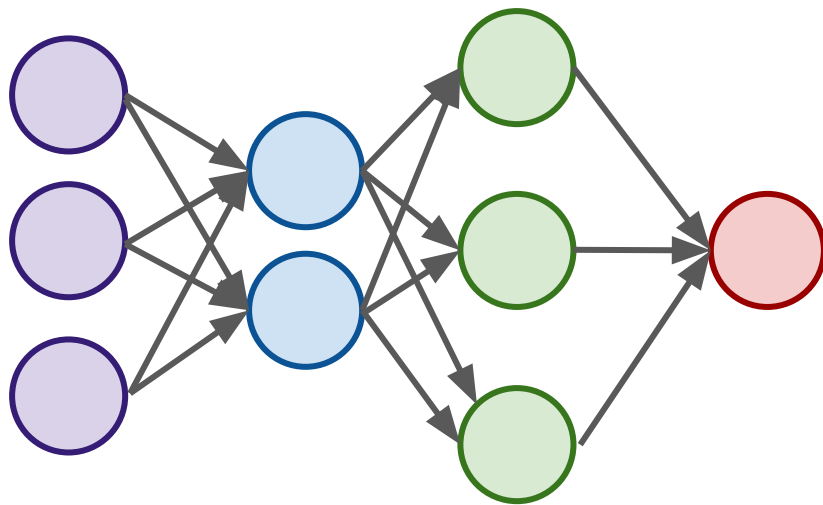
- Un seul perceptron ne suffira pas pour apprendre des systèmes complexes.
- Heureusement, nous pouvons développer l'idée d'un perceptron unique, pour créer un modèle de perceptron multicouche.
- Nous allons également introduire l'idée de fonctions d'activation.





Réseaux de Neurones

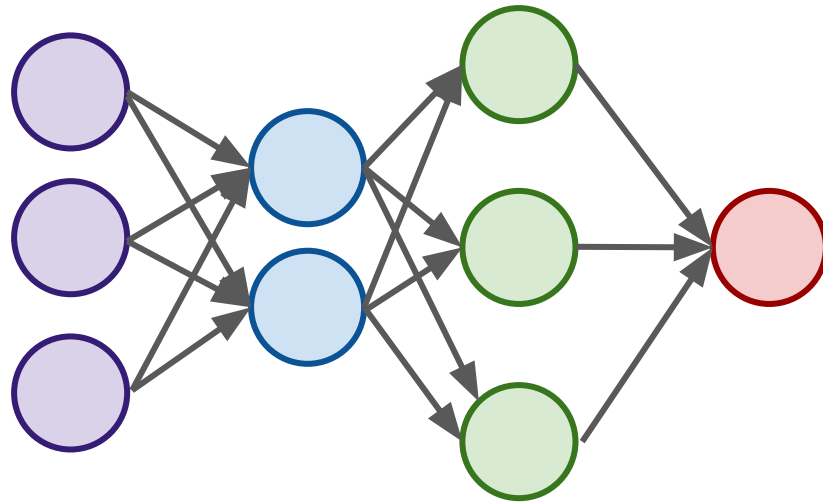
- Pour construire un réseau de perceptrons, nous pouvons connecter des couches de perceptrons, en utilisant un **modèle de perceptron multicouche**.





Réseaux de Neurones

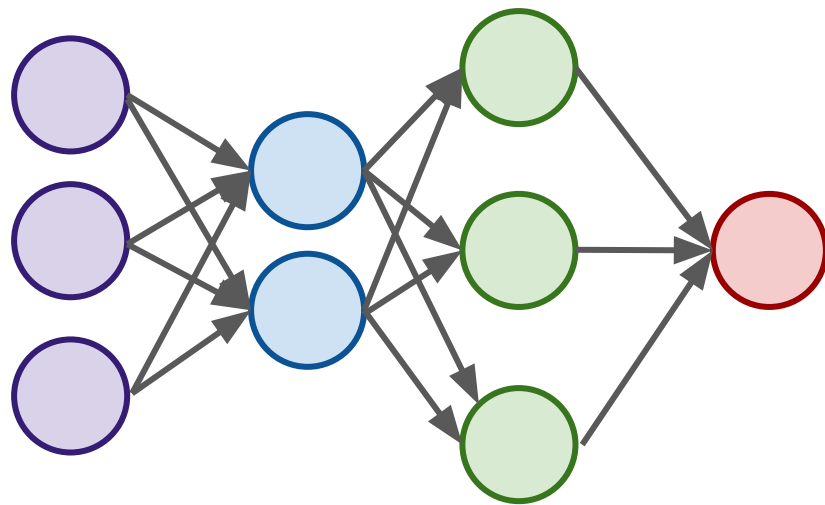
- Les outputs (sorties) d'un perceptron sont directement transmises comme inputs (entrées) à un autre perceptron.





Réseaux de Neurones

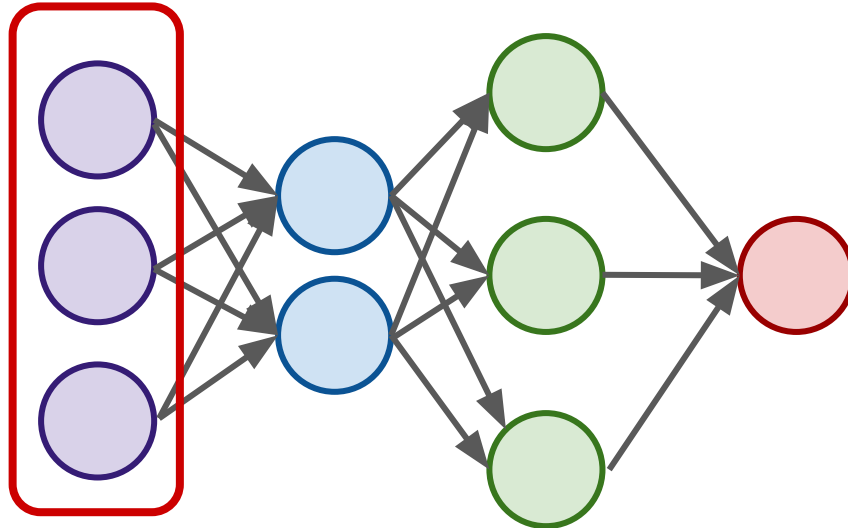
- Cela permet au réseau dans son ensemble de connaître les interactions et les relations entre les features (caractéristiques).





Réseaux de Neurones

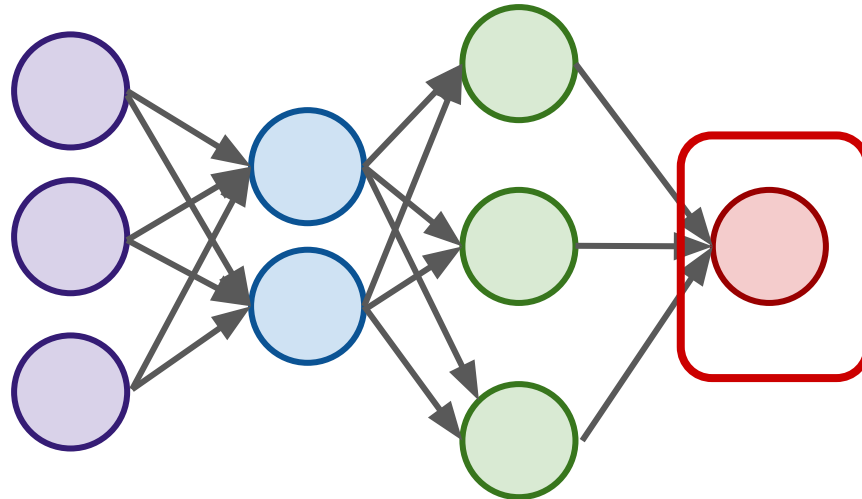
- La première couche est la **input layer** (**couche d'entrée**)





Réseaux de Neurones

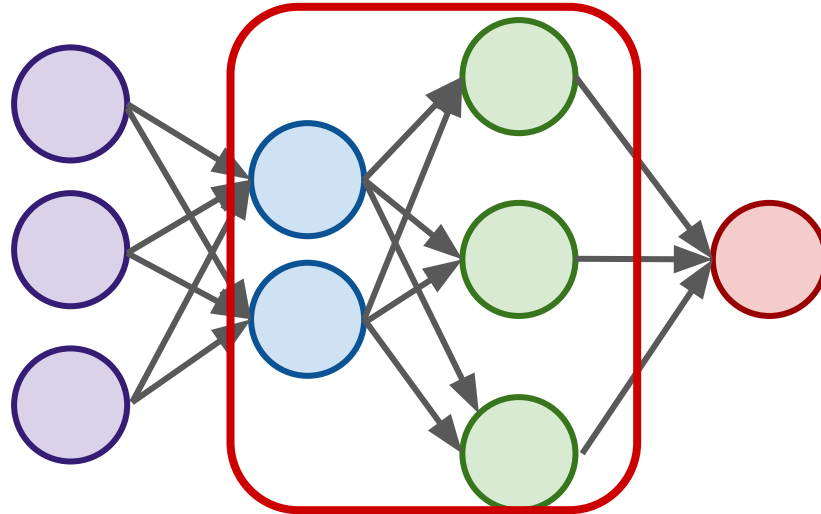
- La dernière couche est la **output layer (couche de sortie)**.
- Note : Cette dernière couche peut être constituée de plus d'un neurone





Réseaux de Neurones

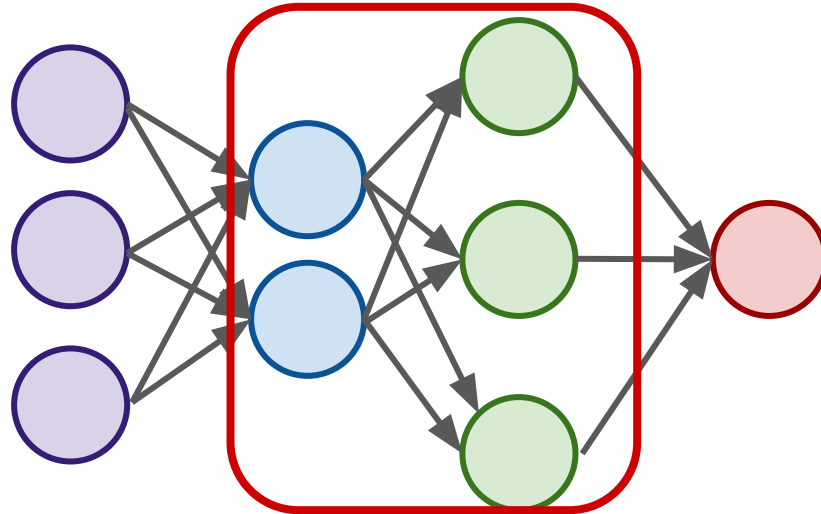
- Les couches situées entre les couches d'entrée et de sortie sont les **hidden layers (couches cachées)**.





Réseaux de Neurones

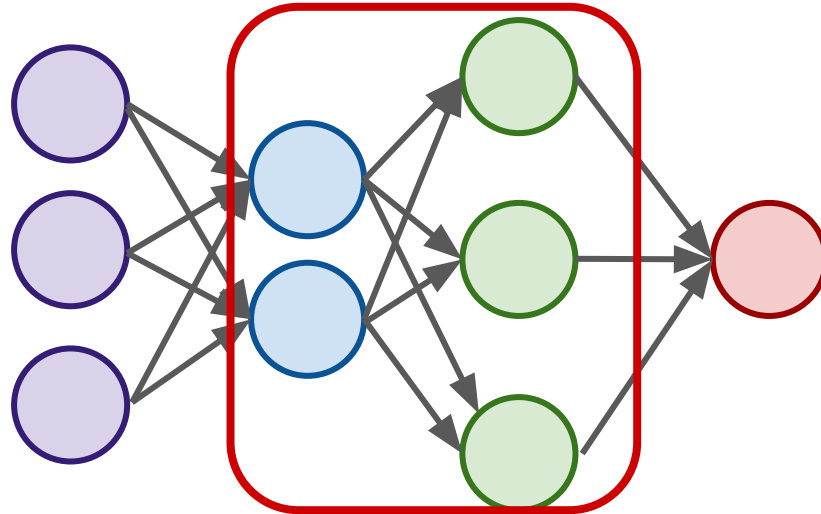
- Les couches cachées sont difficiles à interpréter, en raison de leur forte interconnectivité et de leur éloignement des valeurs d'entrée ou de sortie connues.





Réseaux de Neurones

- Les réseaux de neurones deviennent des **“réseaux de neurones profonds” (“deep neural networks”)** s'ils contiennent au moins 2 couches cachées.

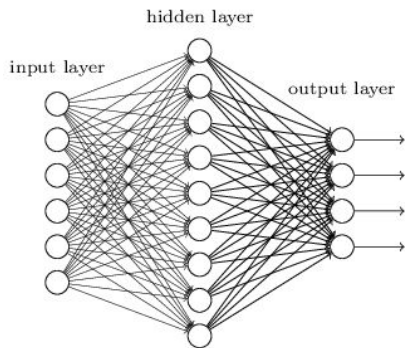




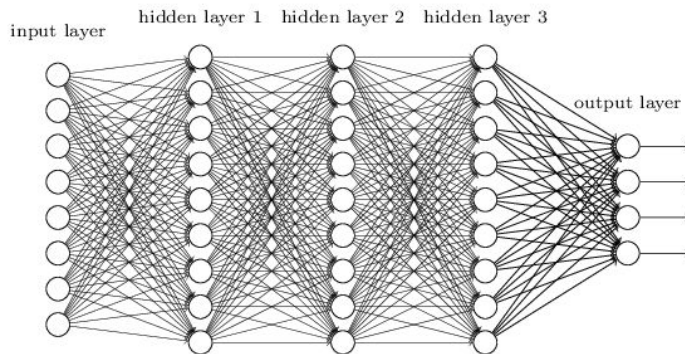
Réseaux de Neurones

- Les réseaux de neurones deviennent des **"réseaux neuronaux profonds"** s'ils contiennent au moins 2 couches cachées.

"Non-deep" feedforward
neural network



Deep neural network





Réseaux de Neurones

- Terminologie :
 - Input Layer (couche d'entrée) : Première couche qui accepte directement les valeurs réelles des données
 - Hidden Layer (couche cachée) : Toute couche entre les couches d'entrée et de sortie
 - Output Layer (couche de sortie) : L'estimation finale de la sortie.





Réseaux de Neurones

- Ce qui est incroyable avec le framework réseaux de neurones, c'est qu'il peut être utilisé pour approximer n'importe quelle fonction.
- Zhou Lu et plus tard Boris Hanin ont prouvé mathématiquement que les réseaux de neurones peuvent approximer n'importe quelle fonction continue convexe.





Réseaux de Neurones

- Pour plus de détails à ce sujet, consultez la page Wikipédia anglaise sur le "Théorème d'approximation universelle".





Réseaux de Neurones

- Auparavant, dans notre modèle simple, nous avons vu que le perceptron lui-même contenait une fonction de somme très simple $f(x)$.
- Pour la plupart des cas d'utilisation, cependant, cela ne sera pas utile, nous voudrions être en mesure de fixer des contraintes à nos valeurs de sortie, en particulier dans les tâches de classification.





Réseaux de Neurones

- Dans une tâche de classification, il serait utile que tous les résultats se situent entre 0 et 1.
- Ces valeurs peuvent ensuite présenter des affectations de probabilité pour chaque classe.
- Dans la prochaine session, nous explorerons comment utiliser les **fonctions d'activation** pour fixer des limites aux valeurs de sortie du neurone.



Fonctions d'Activation



MonCoachData





Réseaux de neurones

- Rappelons que les entrées **x** ont un poids **w** et un terme de biais **b** qui leur est attaché dans le modèle de perceptron.
- Ce qui signifie que nous avons
 - **$x * w + b$**





Réseaux de neurones

- Ce qui signifie que nous avons
 - $\mathbf{x} * \mathbf{w} + \mathbf{b}$
 - Il est clair que $\mathbf{w}$ implique le poids ou la force à donner à la valeur d'entrée.
 - On peut considérer $\mathbf{b}$ comme une valeur de décalage, ce qui fait que $\mathbf{x} * \mathbf{w}$ doit atteindre un certain seuil avant d'avoir un effet.





Réseaux de neurones

- Par exemple si **$b = -10$**
 - **$x * w + b$**
 - Ensuite, les effets de **$x * w$** ne commenceront vraiment à surmonter le biais que lorsque leur produit dépassera 10.
 - Après cela, l'effet est uniquement basé sur la valeur de **w** .
 - D'où le terme de “biais”





Réseaux de neurones

- Ensuite, nous voulons fixer des limites pour la valeur globale de sortie de :
 - $\mathbf{x} * \mathbf{w} + \mathbf{b}$
- Nous pouvons déclarer :
 - $\mathbf{z} = \mathbf{x} * \mathbf{w} + \mathbf{b}$
- Et ensuite faire passer $\mathbf{z}$ par une fonction d'activation pour limiter sa valeur.





Réseaux de neurones

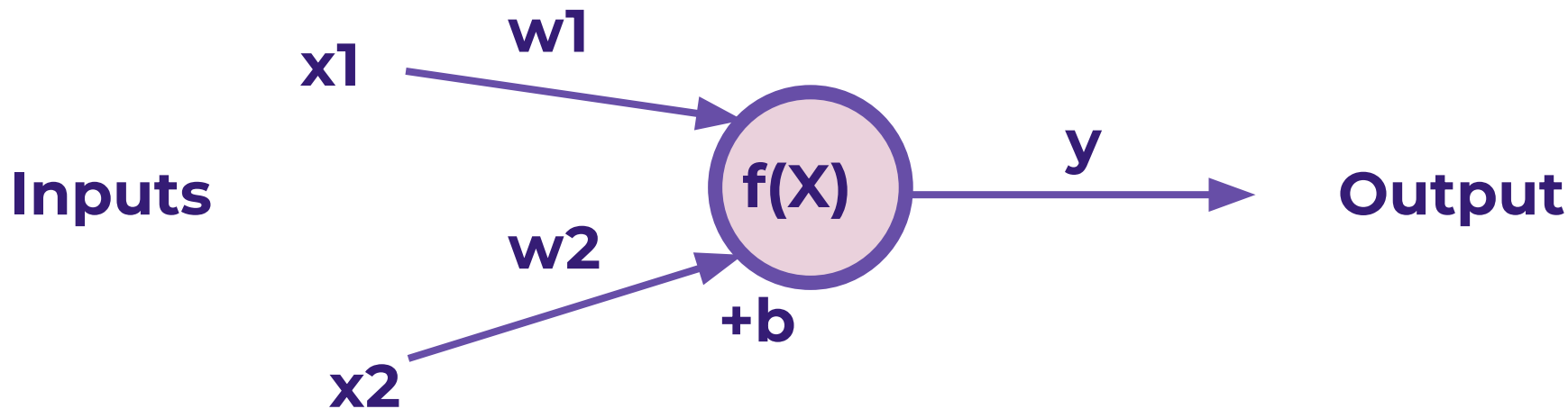
- De nombreuses recherches ont été menées sur les fonctions d'activation et leur efficacité.
- Examinons quelques fonctions d'activation courantes.





Modèle Perceptron

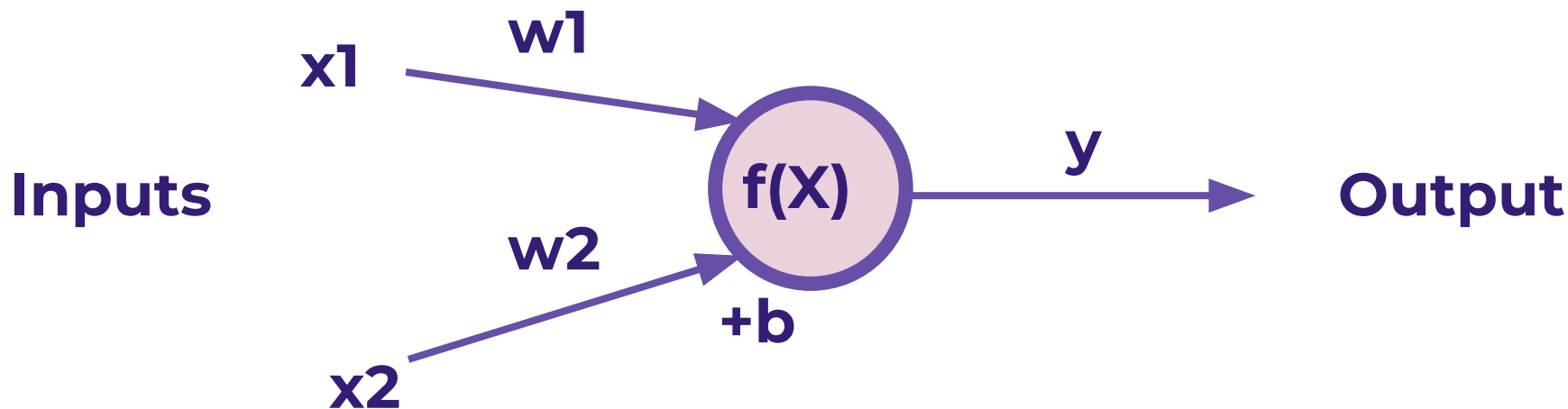
- Rappelons que notre simple perceptron a un $f(X)$





Modèle Perceptron

- Si nous avons un problème de classification binaire, nous voudrions une sortie de 0 ou de 1.





Réseaux de neurones

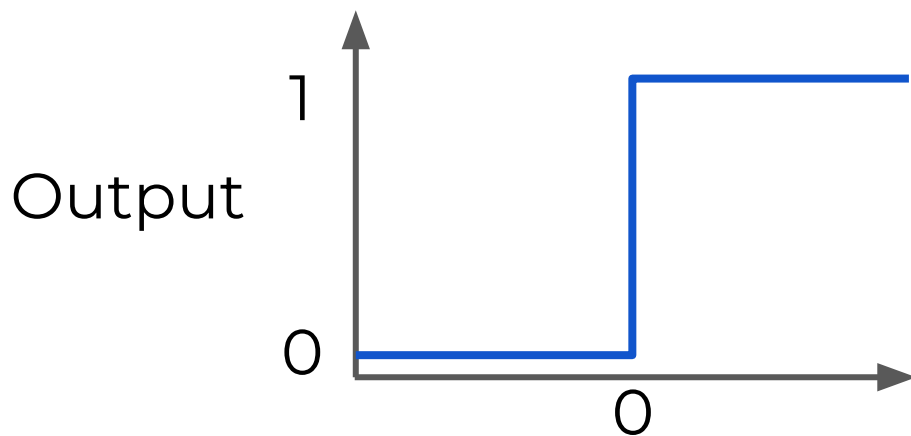
- Pour éviter toute confusion, définissons les entrées totales comme une variable **$\mathbf{z}$** .
- Où **$\mathbf{z} = \mathbf{w}\mathbf{x} + \mathbf{b}$**
- Dans ce contexte, nous appellerons les fonctions d'activation **$\mathbf{f}(\mathbf{z})$** .
- N'oubliez pas que vous verrez souvent ces variables en majuscules **$\mathbf{f}(\mathbf{Z})$** ou **$\mathbf{X}$** pour indiquer une entrée de tenseur composée de valeurs multiples.





Deep Learning

- Les réseaux les plus simples s'appuient sur une **fonction échelon** qui produit 0 ou 1.



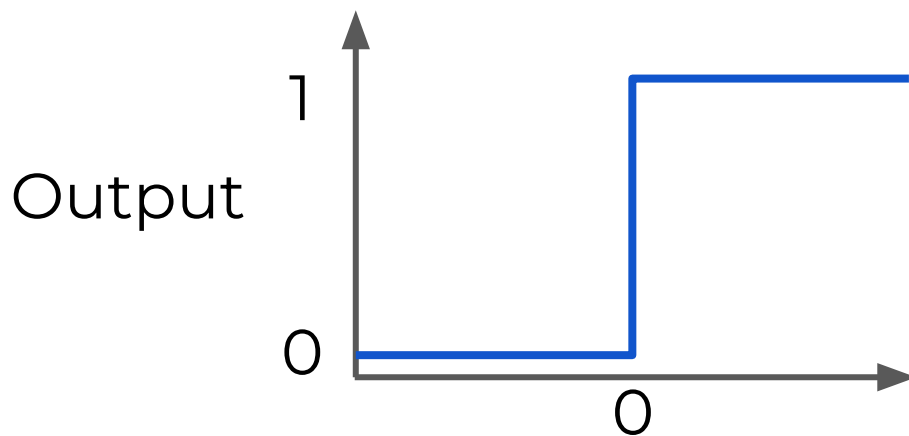
$$z = wx + b$$





Deep Learning

- Quelles que soient les valeurs, cela donne toujours 0 ou 1.



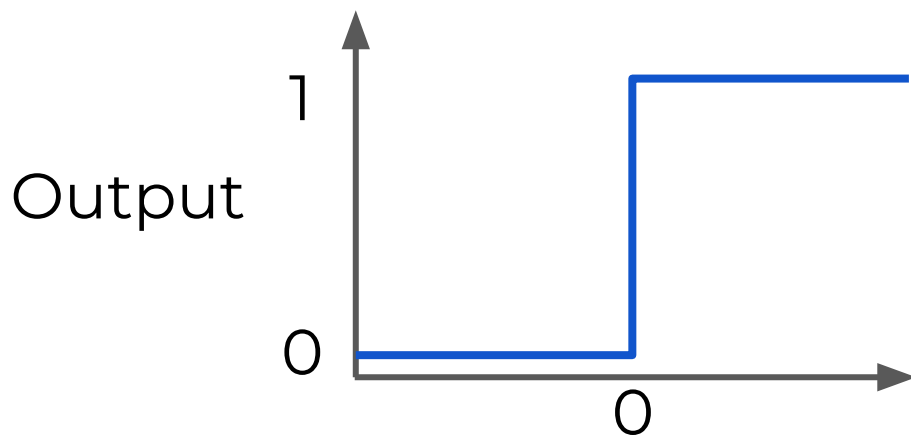
$$z = wx + b$$





Deep Learning

- Ce type de fonction pourrait être utile pour la classification (classe 0 ou 1).



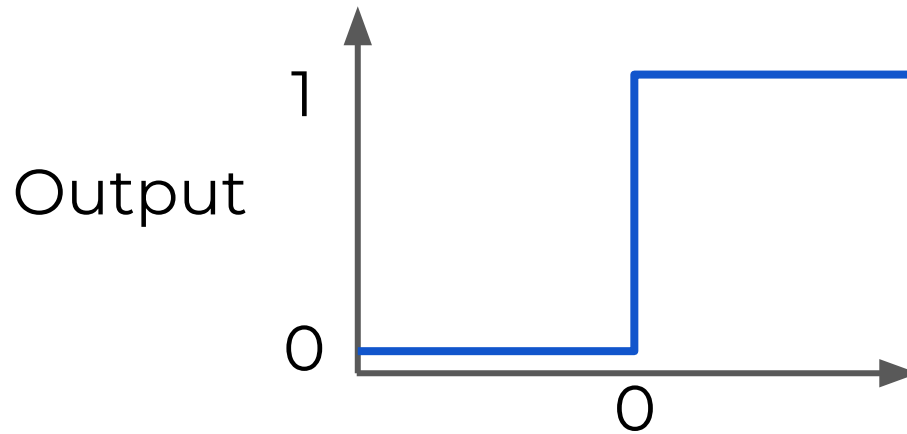
$$z = wx + b$$





Deep Learning

- Cependant, il s'agit d'une fonction très "forte", car les petits changements ne sont pas pris en compte.



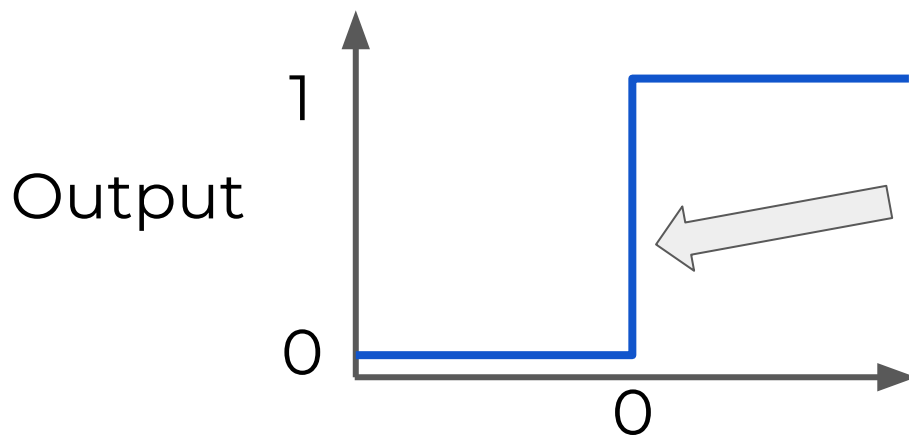
$$z = wx + b$$





Deep Learning

- Il y a juste une coupure immédiate qui sépare 0 et 1.



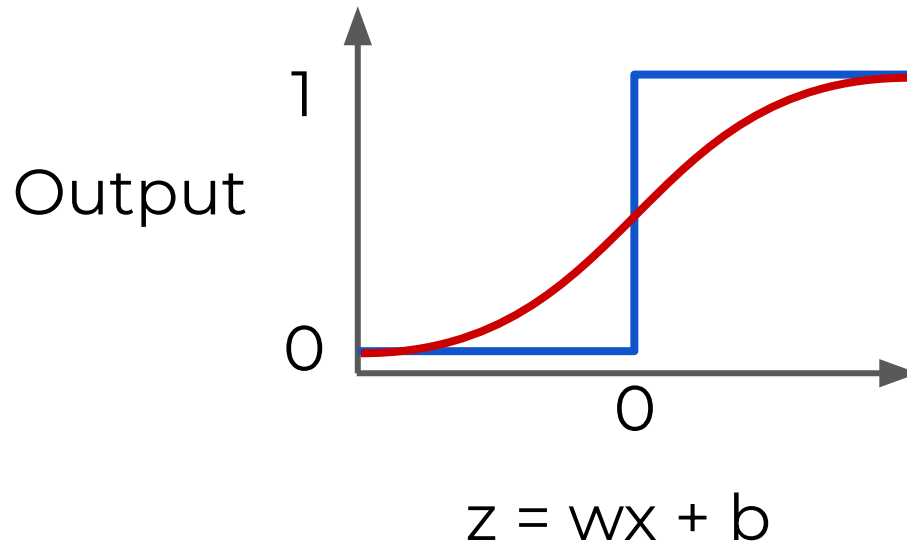
$$z = wx + b$$





Deep Learning

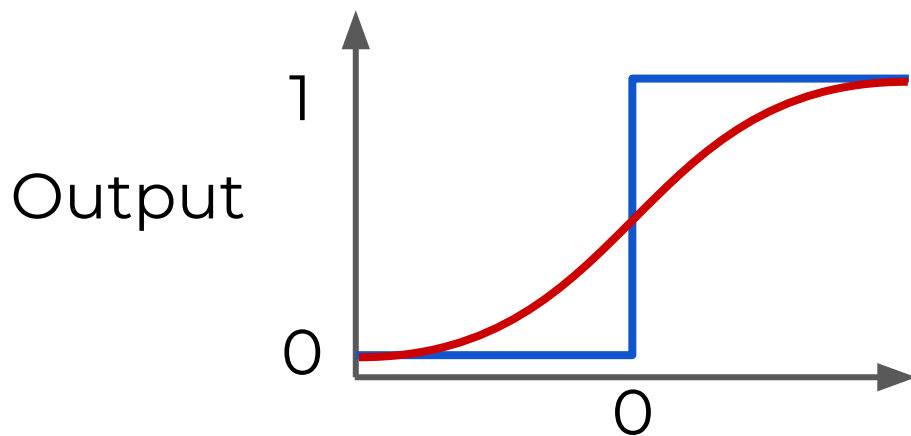
- Ce serait bien si nous pouvions avoir une fonction plus dynamique, par exemple la ligne rouge !





Deep Learning

- Heureusement pour nous, c'est la fonction sigmoïde !

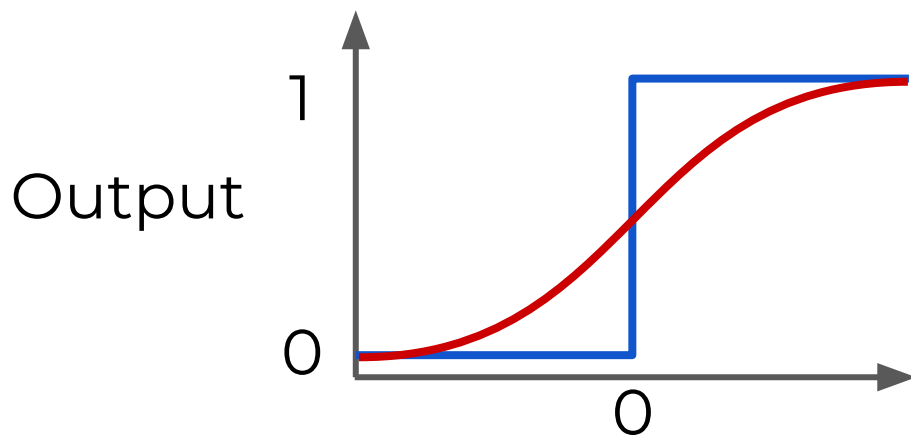


$$f(z) = \frac{1}{1 + e^{(-z)}}$$





- La modification de la fonction d'activation utilisée peut être bénéfique selon la tâche à accomplir !



$$f(z) = \frac{1}{1 + e^{(-z)}}$$

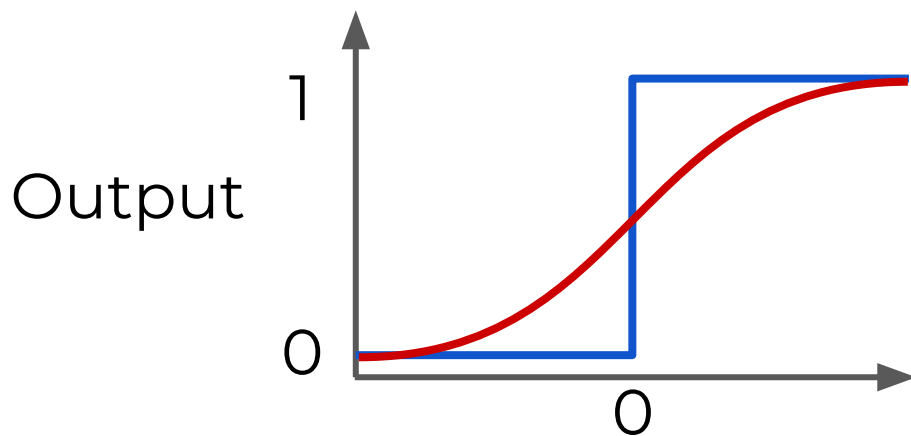
$$z = wx + b$$





Deep Learning

- Cela fonctionne toujours pour la classification, et sera plus sensible aux petits changements.



$$f(z) = \frac{1}{1 + e^{(-z)}}$$

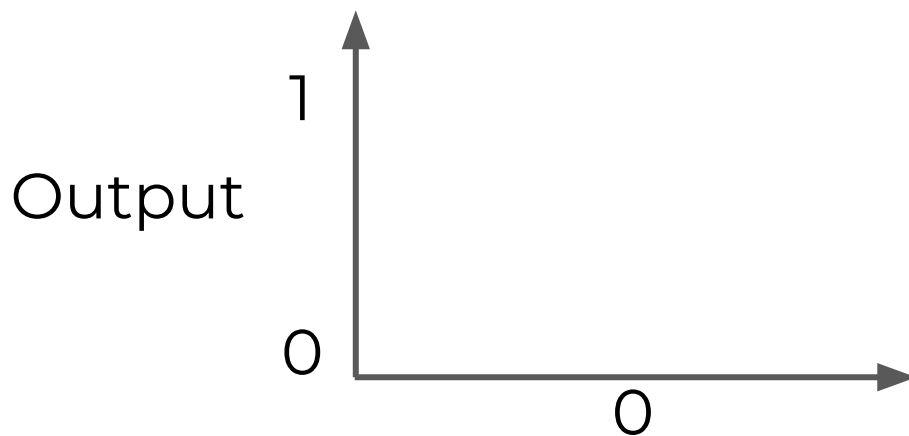
$$z = wx + b$$





Deep Learning

- Discutons de quelques autres fonctions d'activation que nous allons rencontrer !

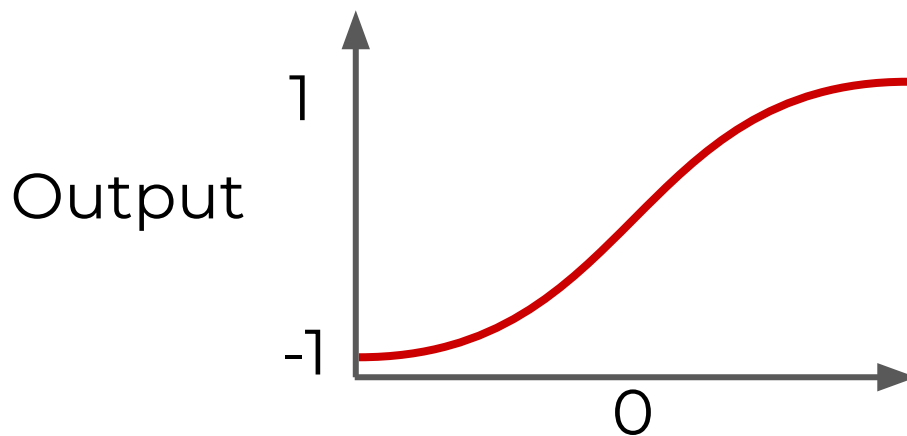


$$z = wx + b$$





- Tangente hyperbolique : $\tanh(z)$



$$\cosh x = \frac{e^x + e^{-x}}{2}$$

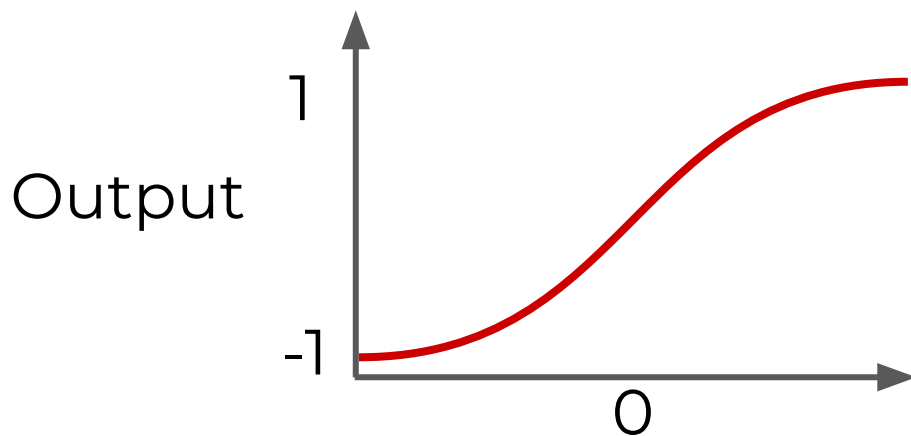
$$\sinh x = \frac{e^x - e^{-x}}{2}$$

$$\tanh x = \frac{\sinh x}{\cosh x}$$





- Tangente hyperbolique : $\tanh(z)$
- Résultats de sortie entre -1 et 1 au lieu de 0 à 1



$$\cosh x = \frac{e^x + e^{-x}}{2}$$

$$\sinh x = \frac{e^x - e^{-x}}{2}$$

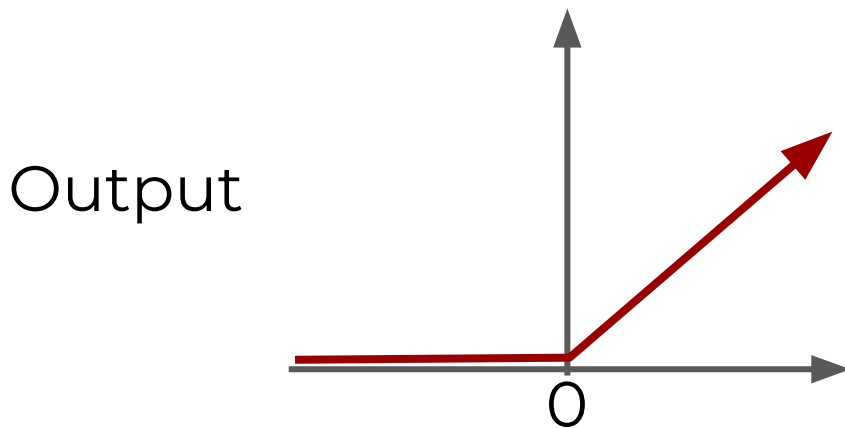
$$\tanh x = \frac{\sinh x}{\cosh x}$$





Deep Learning

- Unité linéaire rectifiée (ReLU) : Il s'agit en fait d'une fonction relativement simple : $\max(0, z)$



$$z = wx + b$$





Deep Learning

- On a constaté que ReLu avait de très bonnes performances, notamment en ce qui concerne la question de **vanishing gradient**.
- Nous choisissons souvent ReLu par défaut en raison de ses bonnes performances générales.





Deep Learning

- Pour une liste complète des fonctions d'activation possibles, consultez :
 - **en.wikipedia.org/wiki/Activation_function**



Fonctions d'Activation Multi-Classe





Deep Learning

- Remarquez que toutes ces fonctions d'activation ont un sens pour une seule sortie, que ce soit une étiquette (label) continue ou une tentative de prédire une classification binaire (soit un 0 ou un 1).
- Mais que devons-nous faire si nous avons une situation multiclasse ?





Deep Learning

- Il existe deux grands types de situations multiclass
 - Classes non-exclusives
 - Un point de données peut se voir attribuer plusieurs classes/catégories
 - Classes mutuellement exclusives
 - Une seule classe par point de données.



Deep Learning

- Classes non-exclusives
 - Un point de données peut se voir attribuer plusieurs classes/catégories
 - Les photos peuvent avoir plusieurs tags (par exemple plage, famille, vacances, etc...)





Deep Learning

- Classes mutuellement exclusives
 - Un point de données ne peut se voir attribuer qu'une seule classe/catégorie
 - Les photos peuvent être classées comme étant en niveaux de gris (noir et blanc) ou en couleur. Une photo ne peut pas être les deux à la fois.





Deep Learning

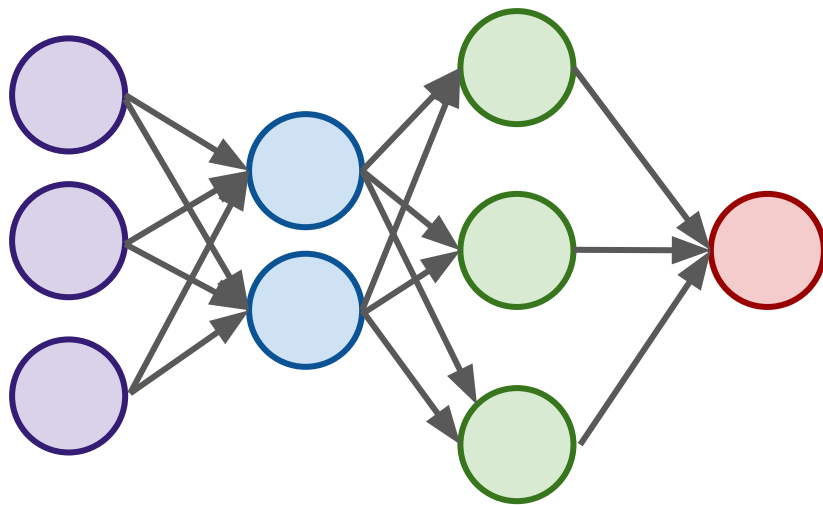
- Organisation des classes multiples
 - La façon la plus simple d'organiser plusieurs classes est de disposer simplement d'un nœud de sortie par classe.





Réseaux de neurones

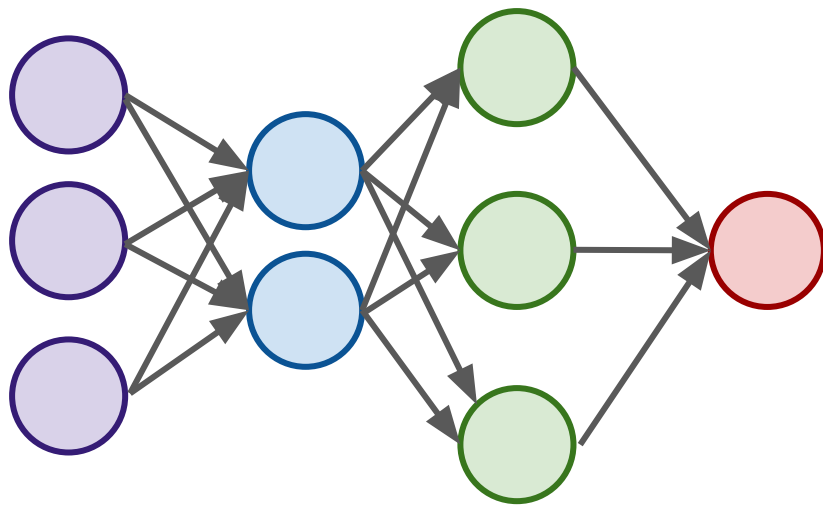
- Auparavant, nous considérons la dernière couche de sortie comme un seul nœud.





Réseaux de neurones

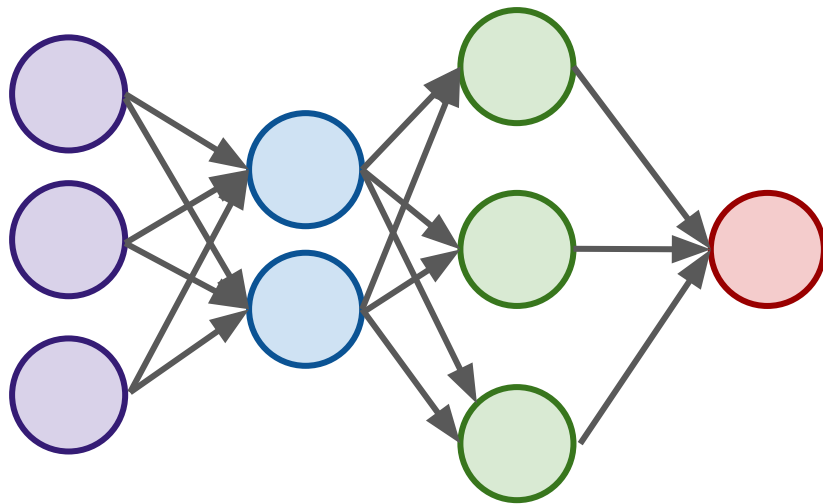
- Ce nœud unique pourrait produire une valeur de régression continue ou une classification binaire (0 ou 1).





Réseaux de neurones

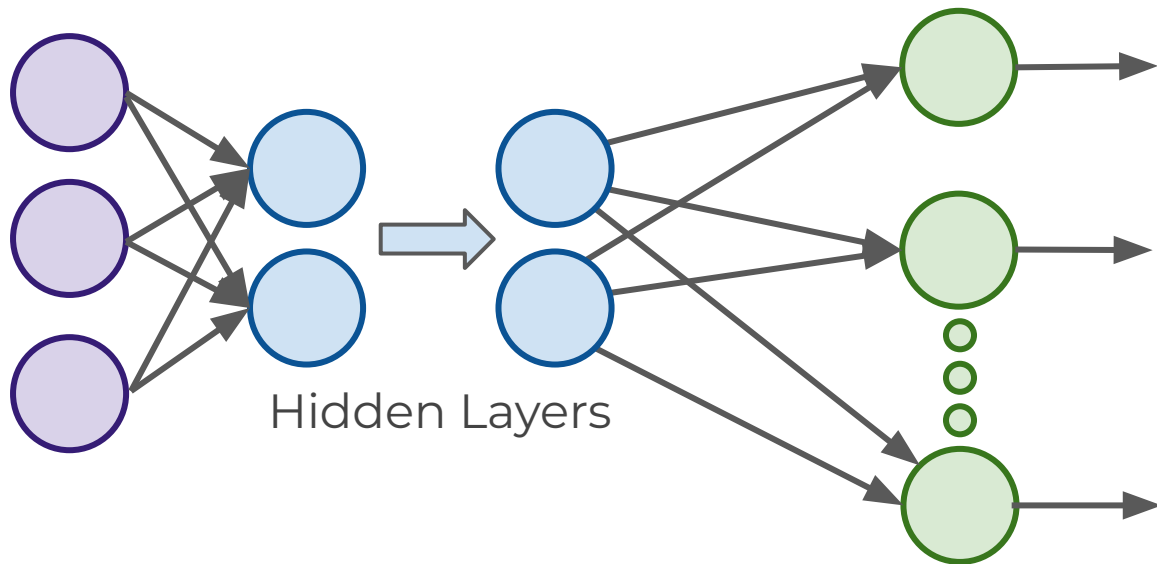
- Étendons cette couche de sortie pour travailler dans le cas de la multi-classification.





Classification Multiclasse

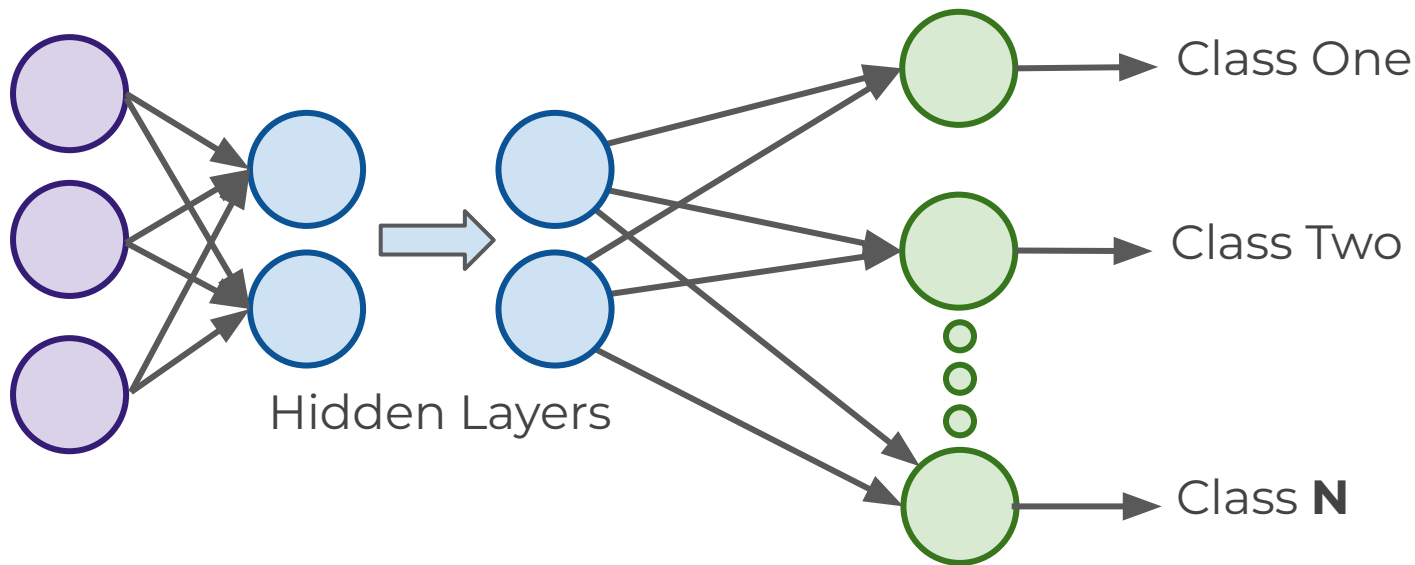
- Organisation pour plusieurs classes





Classification Multiclasse

- Organisation pour plusieurs classes





Deep Learning

- Organisation des classes multiples
 - Cela signifie que nous devons organiser des catégories pour cette couche de sortie.
 - On ne peut pas avoir des catégories comme "rouge", "bleu", "vert", etc...





- Organisation des classes multiples
 - Nous utilisons à la place le **one-hot encoding**
 - Voyons à quoi cela ressemble pour les classes mutuellement exclusives.





- Classes mutuellement exclusives

Data Point 1	RED
Data Point 2	GREEN
Data Point 3	BLUE
...	...
Data Point N	RED





- Classes mutuellement exclusives

Data Point 1	RED
Data Point 2	GREEN
Data Point 3	BLUE
...	...
Data Point N	RED



	RED	GREEN	BLUE
Data Point 1	1	0	0
Data Point 2	0	1	0
Data Point 3	0	0	1
...	...	...	...
Data Point N	1	0	0





- Classes non-exclusives

Data Point 1	A,B
Data Point 2	A
Data Point 3	C,B
...	...
Data Point N	B



	A	B	C
Data Point 1	1	1	0
Data Point 2	1	0	0
Data Point 3	0	1	1
...	...	...	...
Data Point N	0	1	0





Deep Learning

- Maintenant que nos données sont correctement organisées, il nous suffit de choisir la bonne fonction d'activation de classification que la dernière couche de sortie devrait avoir.





Deep Learning

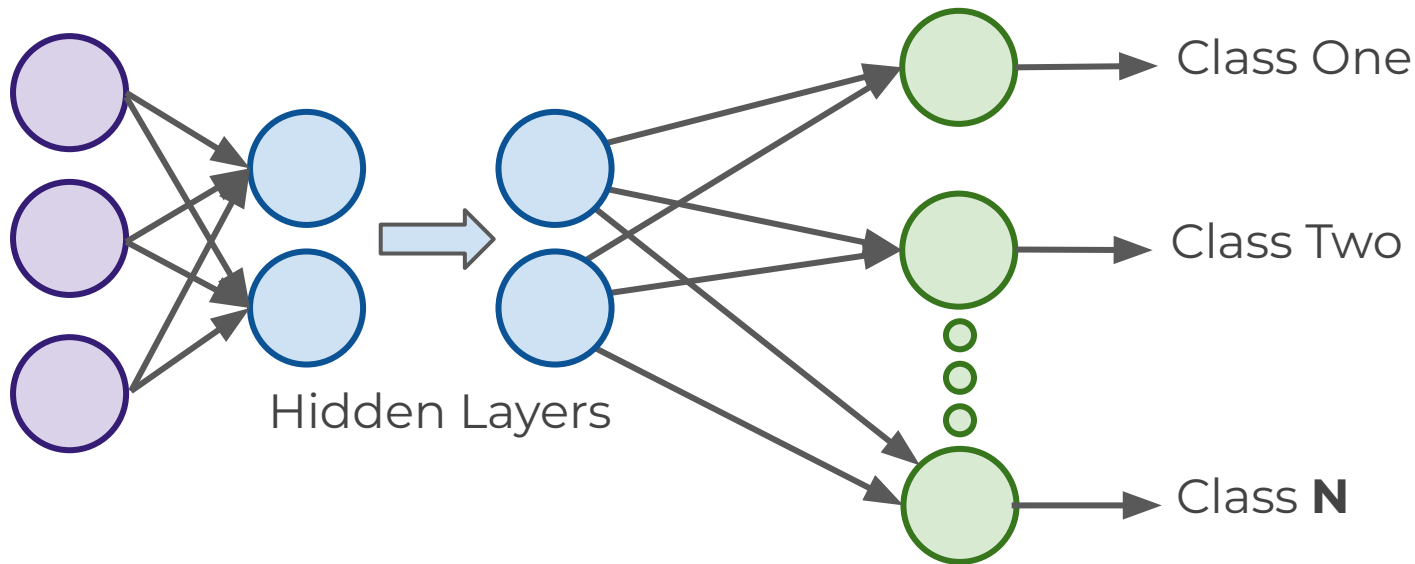
- Non-exclusive
 - Fonction sigmoïde
 - Chaque neurone produira une valeur de sortie entre 0 et 1, indiquant la probabilité de se voir attribuer cette classe.





Classification Multiclasse

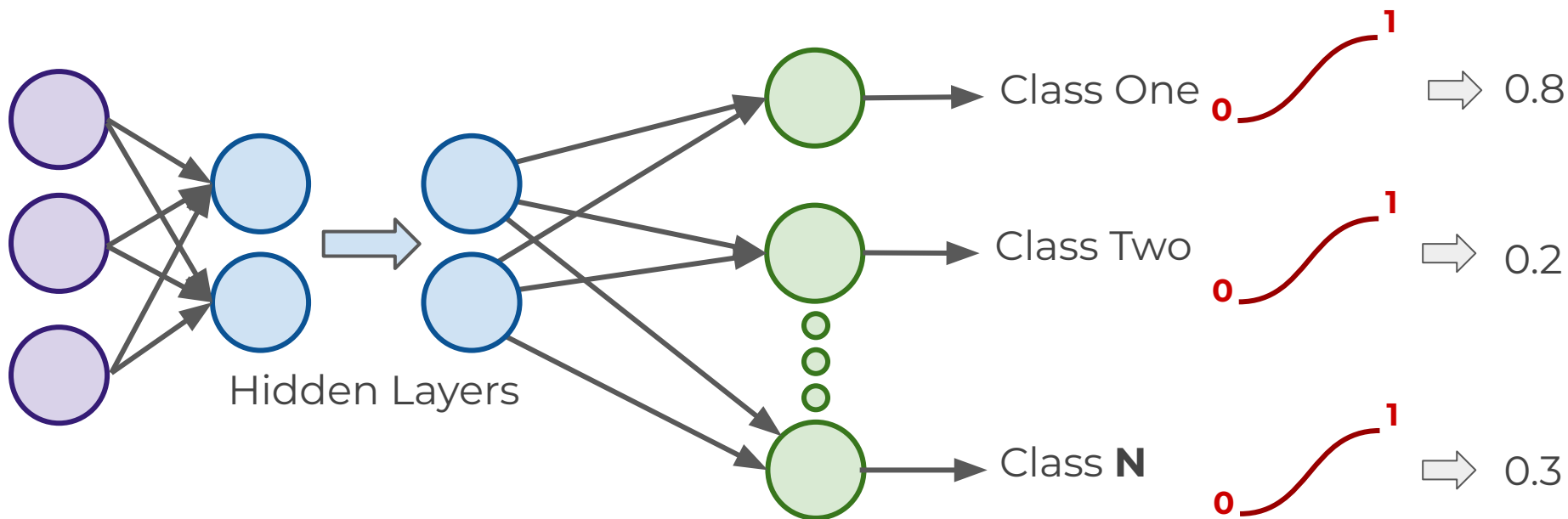
- Fonction Sigmoidale pour les classes non exclusives





Classification Multiclasse

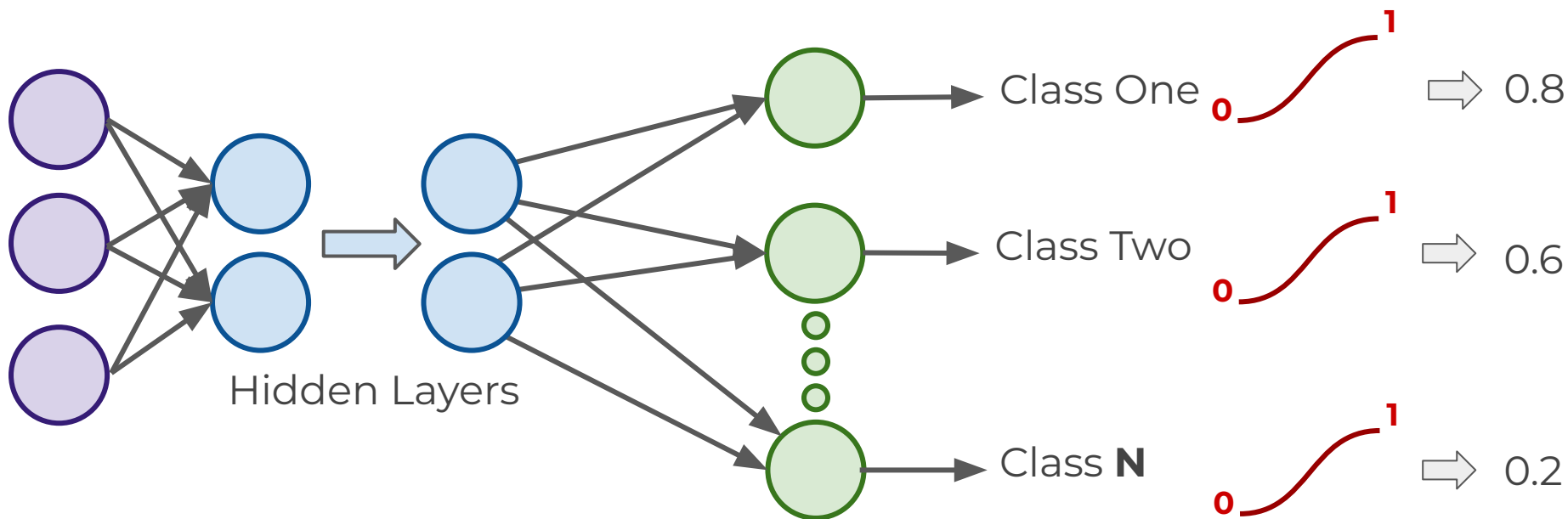
- Fonction Sigmoidé pour les classes non exclusives





Classification Multiclasse

- Fonction Sigmoidé pour les classes non exclusives





Deep Learning

- Non-exclusive
 - Fonction sigmoïde
 - Gardez à l'esprit que cela permet à chaque neurone de produire une sortie indépendamment des autres classes, ce qui permet à un seul point de données alimenté dans la fonction de se voir attribuer plusieurs classes.





Deep Learning

- Classes mutuellement exclusives
 - Mais que faire lorsque chaque point de données ne peut se voir attribuer qu'une seule classe ?
 - Nous pouvons utiliser la **fonction softmax** pour cela !





- Fonction Softmax

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \text{for } i = 1, \dots, K$$





Deep Learning

- Classes mutuellement exclusives
 - La fonction Softmax calcule la distribution des probabilités de l'événement sur **K** événements différents.
 - Cette fonction calcule les probabilités de chaque classe de cible sur toutes les classes de cibles possibles.





Deep Learning

- Classes mutuellement exclusives
 - L'intervalle sera de 0 à 1, et **la somme de toutes les probabilités sera égale à un.**
 - Le modèle retourne les probabilités de chaque classe et la classe cible choisie aura la probabilité la plus élevée.





Deep Learning

- Classes mutuellement exclusives
 - La chose principale à garder à l'esprit est que si vous utilisez softmax pour des problèmes multi-classes, vous obtenez ce genre de résultat :
 - [Red , Green , Blue]
 - [0.1 , 0.6 , 0.3]





Deep Learning

- Classes mutuellement exclusives
 - Les probabilités pour chaque classe s'élèvent toutes à 1. Nous choisissons la probabilité la plus élevée pour notre mission.
 - [Red , Green , Blue]
 - [0.1 , 0.6 , 0.3]





Deep Learning

- Récap
 - Les perceptrons étendus au modèle de réseau de neurones
 - Poids et Biais
 - Fonctions d'Activation
 - Il est temps de voir les fonctions de Coût d'erreur (Cost) !



Fonctions de Coût et Descente de Gradient



MonCoachData



Deep Learning

- Nous comprenons maintenant que les réseaux de neurones absorbent des données, les multiplient par des poids et leur ajoutent des biais.
- Ce résultat passe ensuite par une fonction d'activation qui, à la fin de toutes les couches, mène à une sortie.





Deep Learning

- Cette sortie (output) $\hat{y}$ est l'estimation du modèle de ce qu'il prédit pour l'étiquette (label).
- Donc, une fois que le réseau a créé sa prédiction, comment l'évaluer ?
- Et après l'évaluation, comment pouvons-nous actualiser les poids et les biais du réseau ?





Deep Learning

- Nous devons prendre les résultats estimés du réseau et les comparer ensuite aux valeurs réelles du label.
- Gardez à l'esprit qu'il s'agit d'utiliser l'ensemble des données d'entraînement pendant l'ajustement/entraînement du modèle.





Deep Learning

- La fonction de coût, souvent appelée fonction de perte (loss) doit être une moyenne afin de pouvoir produire une valeur unique.
- Nous pouvons suivre nos pertes/coûts pendant l'entraînement pour contrôler les performances du réseau.





Deep Learning

- Nous utiliserons les variables suivantes :
 - **y** pour représenter la valeur réelle
 - **a** pour représenter la prédiction du neurone
- En termes de poids et de biais :
 - $w * x + b = z$
 - Passez z en fonction d'activation $\sigma(z) = a$





Deep Learning

- Une fonction de coût très courante est la fonction de coût quadratique :

$$C = \frac{1}{2n} \sum_x \|y(x) - a^L(x)\|^2$$





Deep Learning

- Nous calculons simplement la différence entre les valeurs réelles $y(x)$ et nos valeurs prédites $a(x)$.

$$C = \frac{1}{2n} \sum_x \|y(x) - a^L(x)\|^2$$



Deep Learning

- Note : La notation indiquée ici correspond aux entrées et sorties de vecteurs, puisque nous aurons affaire à un **batch** de points d'entraînement et de prédictions.

$$C = \frac{1}{2n} \sum_x \|y(x) - a^L(x)\|^2$$



Deep Learning

- Remarquez comme le carré de tout ça fait 2 choses utiles pour nous, garde tout positif et **punit** les grosses erreurs !

$$C = \frac{1}{2n} \sum_x \|y(x) - a^L(x)\|^2$$



Deep Learning

- On peut considérer la fonction de coût comme :

$$C(W, B, S^r, E^r)$$





Deep Learning

- **W** est le poids de notre réseau de neurones, **B** est le biais de notre réseau de neurones, **S^r** est l'apport d'un seul échantillon d'entraînement en entrée, et **E^r** est le résultat souhaité en sortie de cet échantillon d'entraînement.

$$C(W, B, S^r, E^r)$$





Deep Learning

- Remarquez comment toutes ces informations ont été encodées dans notre notation simplifiée.
- Le **$\mathbf{a}(\mathbf{x})$** contient des informations sur les poids et les biais.

$$C = \frac{1}{2n} \sum_x \|y(x) - a^L(x)\|^2$$



Deep Learning

- Cela signifie que si nous disposons d'un vaste réseau, nous pouvons nous attendre à ce que $\mathbf{C}$ soit assez complexe, avec d'énormes vecteurs de poids et de biais.

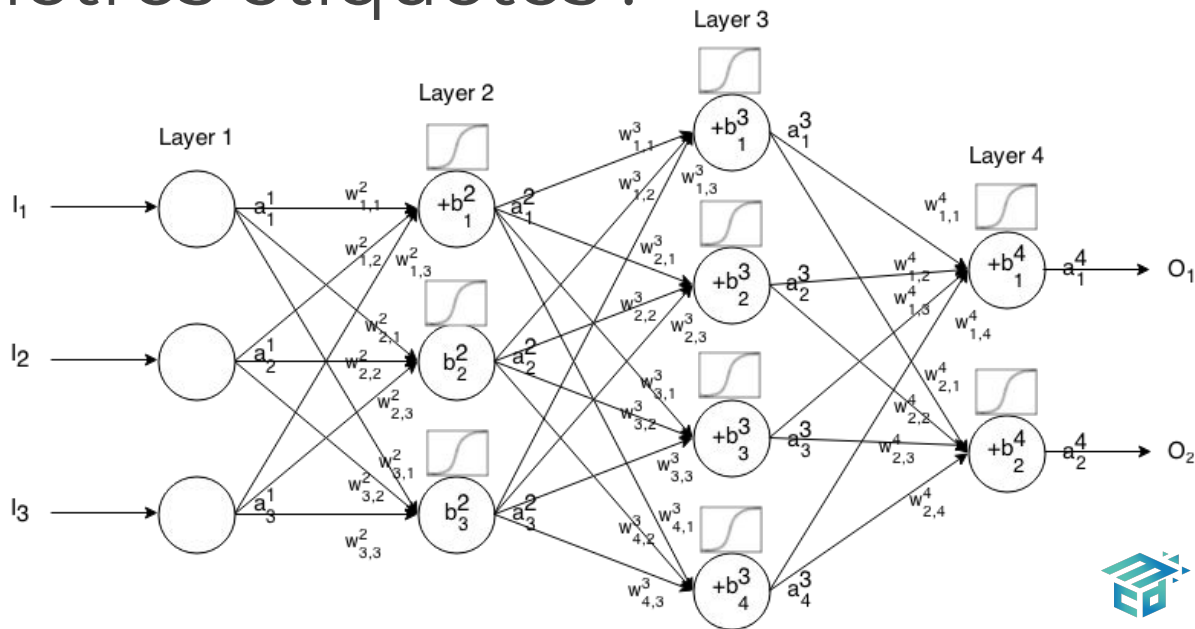
$$C(W, B, S^r, E^r)$$





Deep Learning

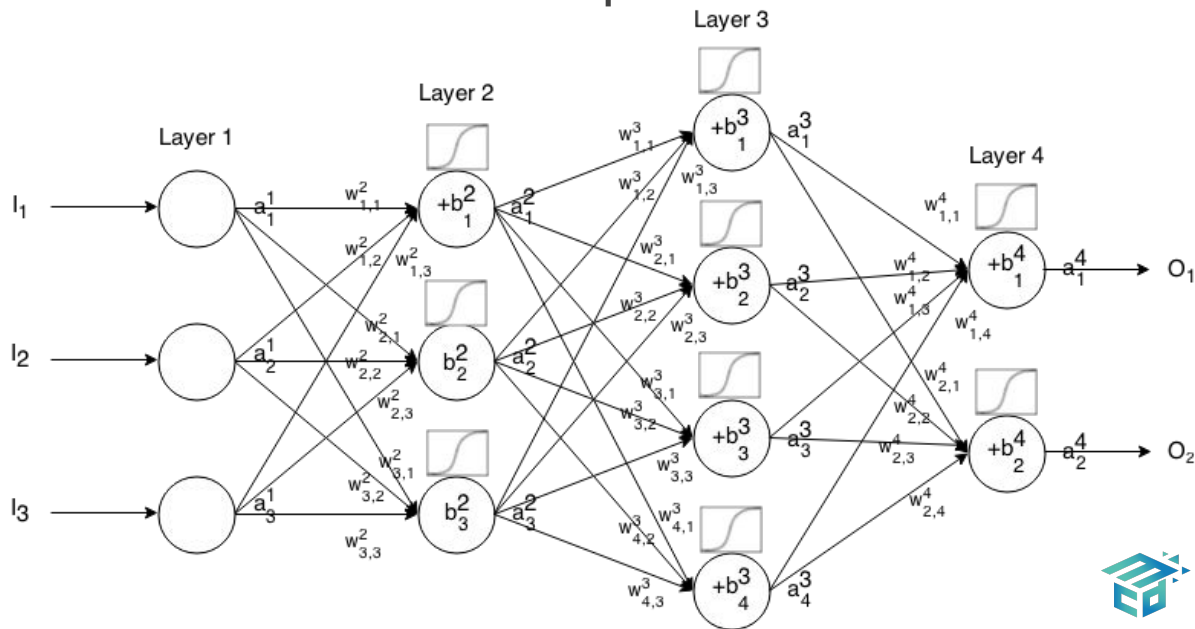
- Voici un petit réseau avec tous ses paramètres étiquetés :





Deep Learning

- Ça fait beaucoup à calculer !
- Comment résoudre ce problème ?





Deep Learning

- Dans un cas réel, cela signifie que nous avons une fonction de coût **C** dépendant de nombreux poids !
 - **$C(w_1, w_2, w_3, \dots, w_n)$**
- Comment déterminer quels poids nous conduisent au coût le plus bas ?





Deep Learning

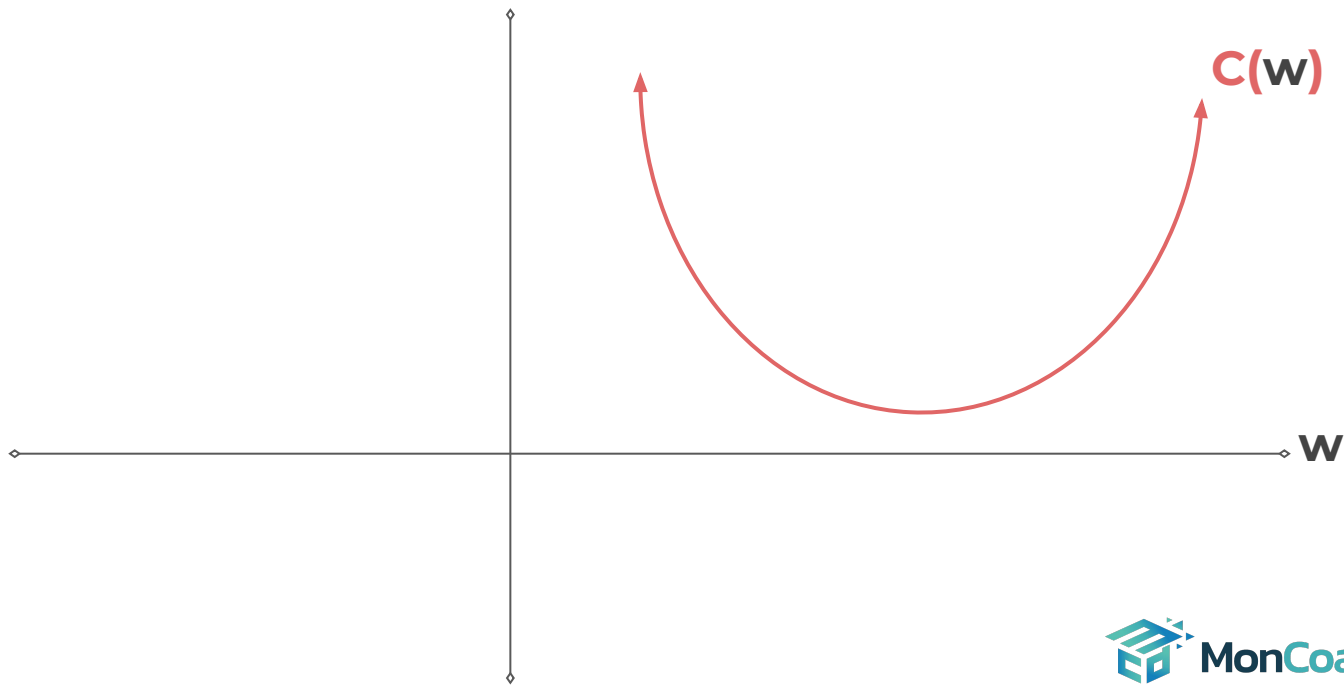
- Pour simplifier, imaginons que nous n'ayons qu'un seul poids dans notre fonction de coût **w** .
- Nous voulons **minimiser** notre perte / coût (erreur globale).
- Ce qui signifie que nous devons déterminer quelle valeur de **w** donne le minimum de **$C(w)$**





Deep Learning

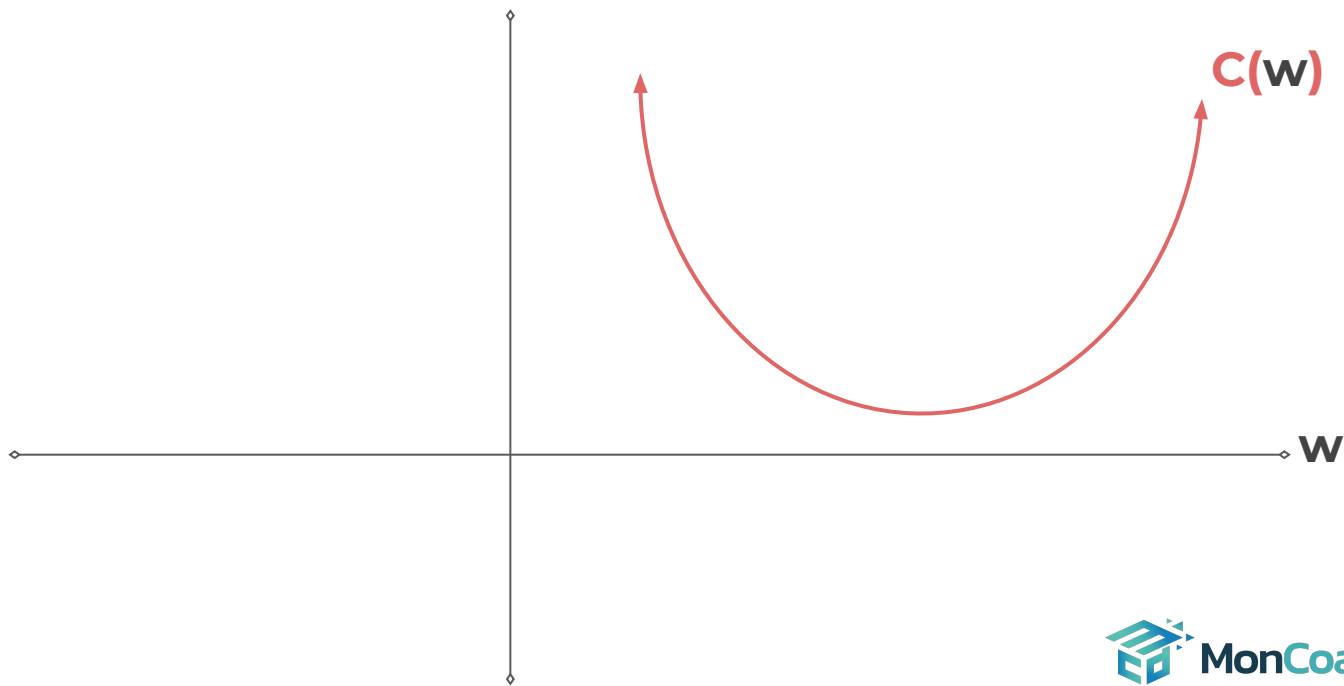
- Notre fonction “simple” $C(w)$





Deep Learning

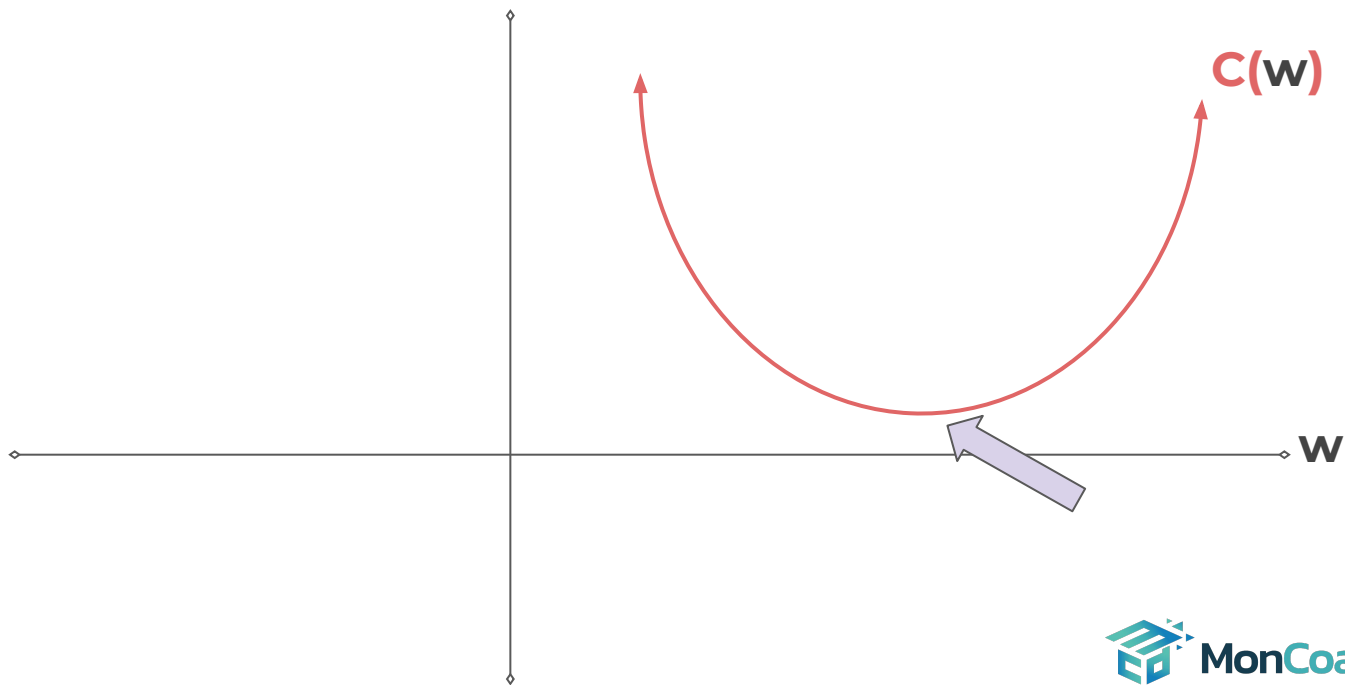
- Quelle est la valeur de $\mathbf{w}$ qui minimise notre coût ?





Deep Learning

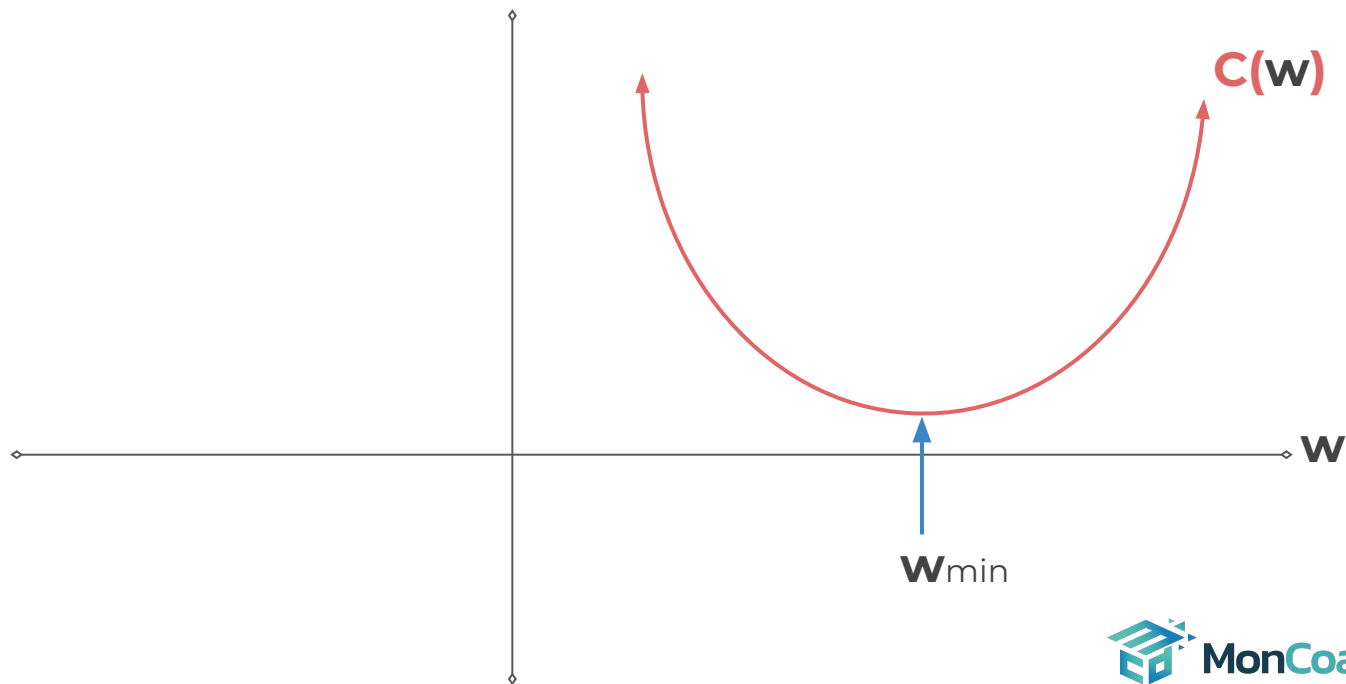
- Quelle est la valeur de **w** qui minimise notre coût ?





Deep Learning

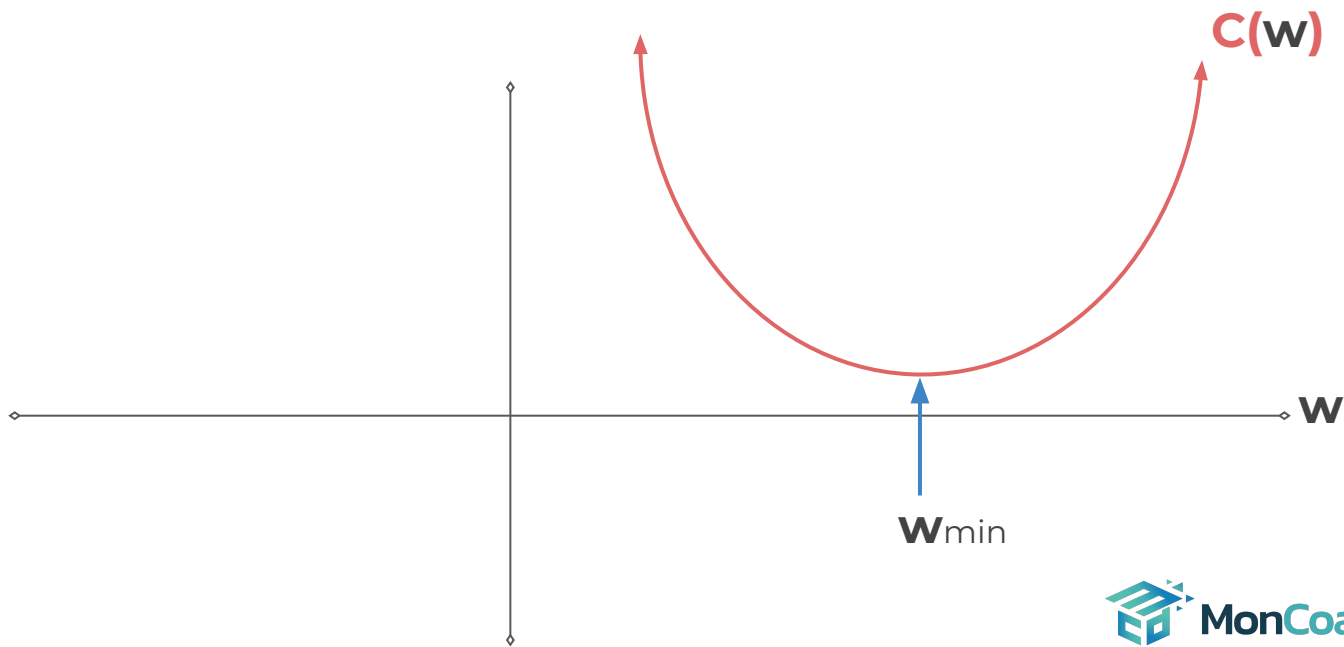
- Quelle est la valeur de $\mathbf{w}$ qui minimise notre coût ?





Deep Learning

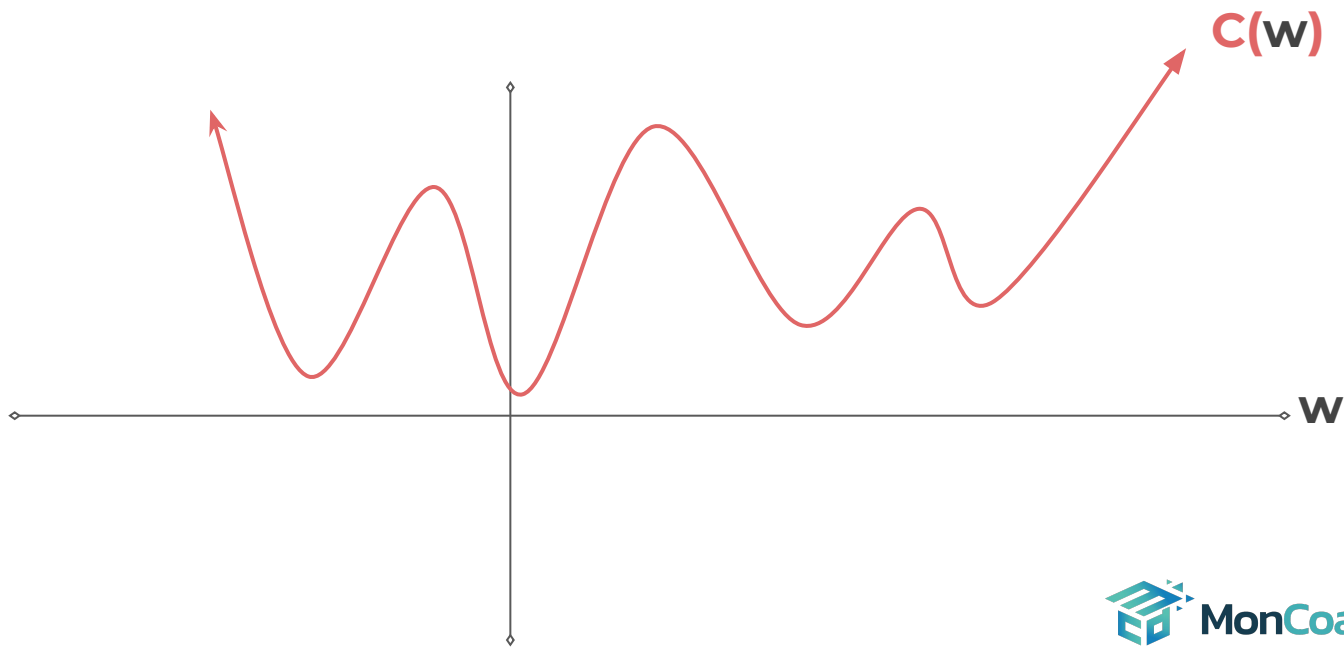
- Les étudiants en calcul savent que nous pourrions prendre une dérivée et résoudre pour 0.





Deep Learning

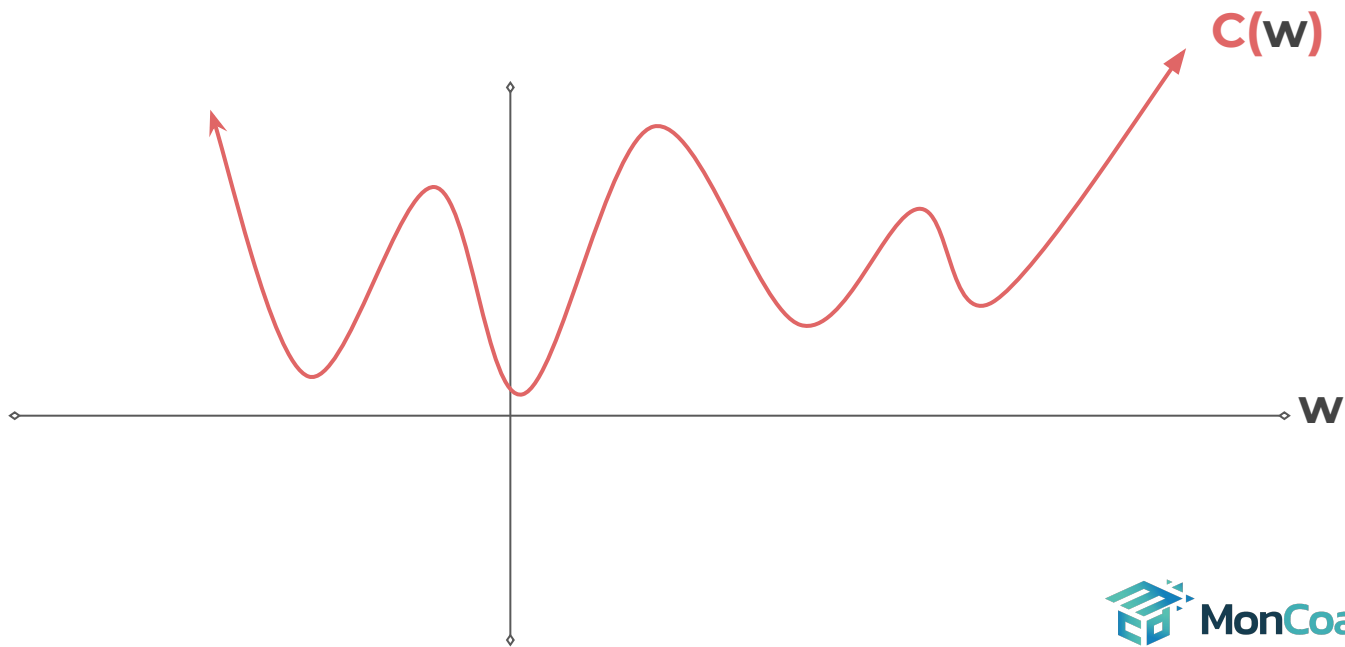
- Mais rappelez-vous que notre fonction de coût réel sera très complexe !





Deep Learning

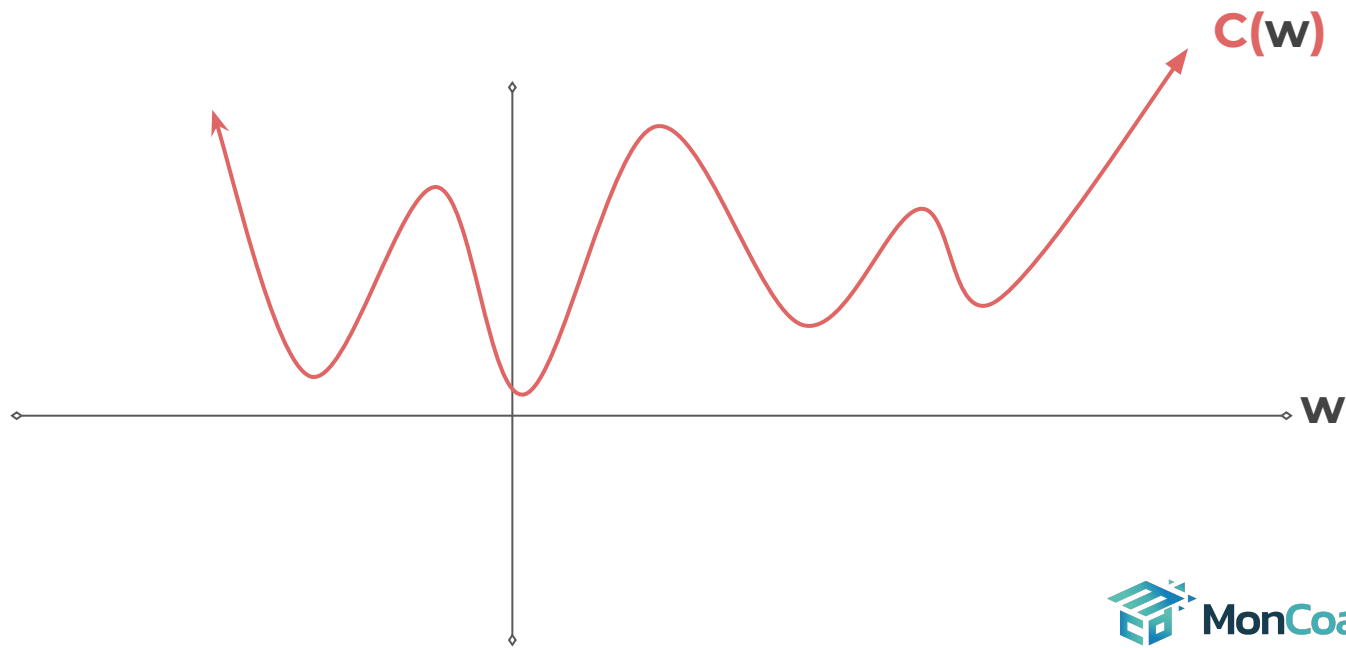
- Et elle sera **n-dimensionnelle** !





Deep Learning

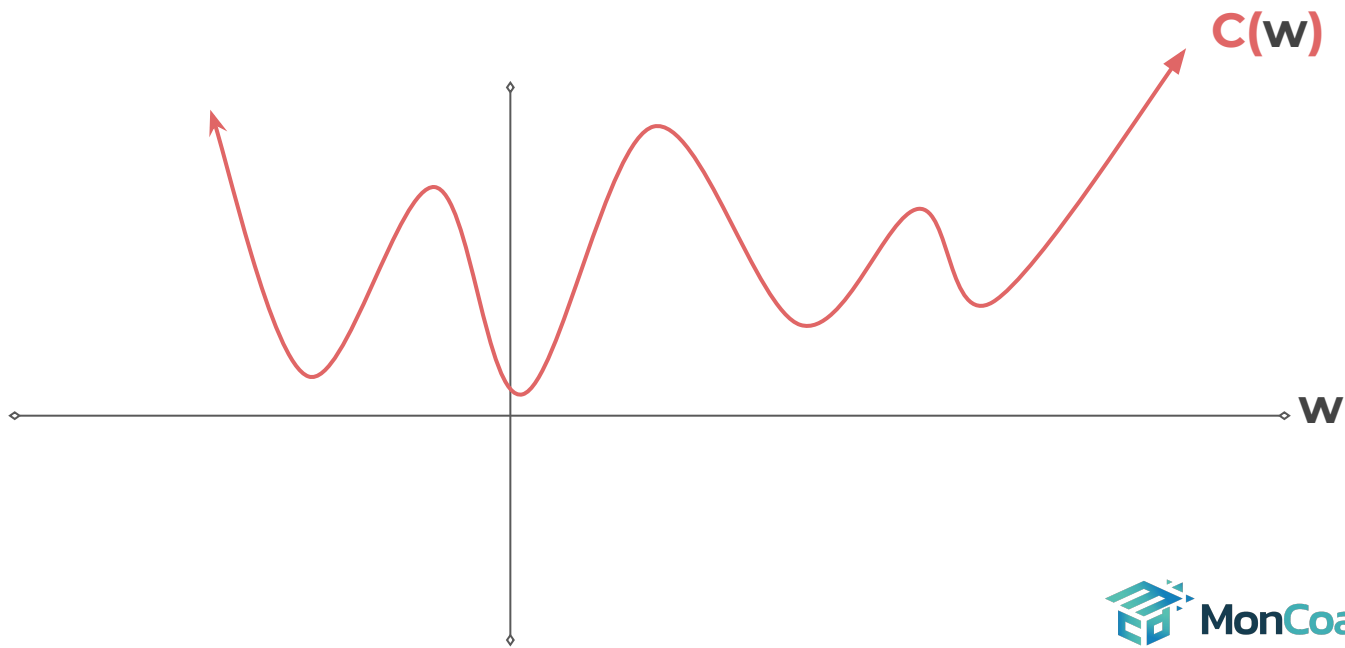
- Et elle sera **n-dimensionnelle** puisque nos réseaux auront des milliers de poids





Deep Learning

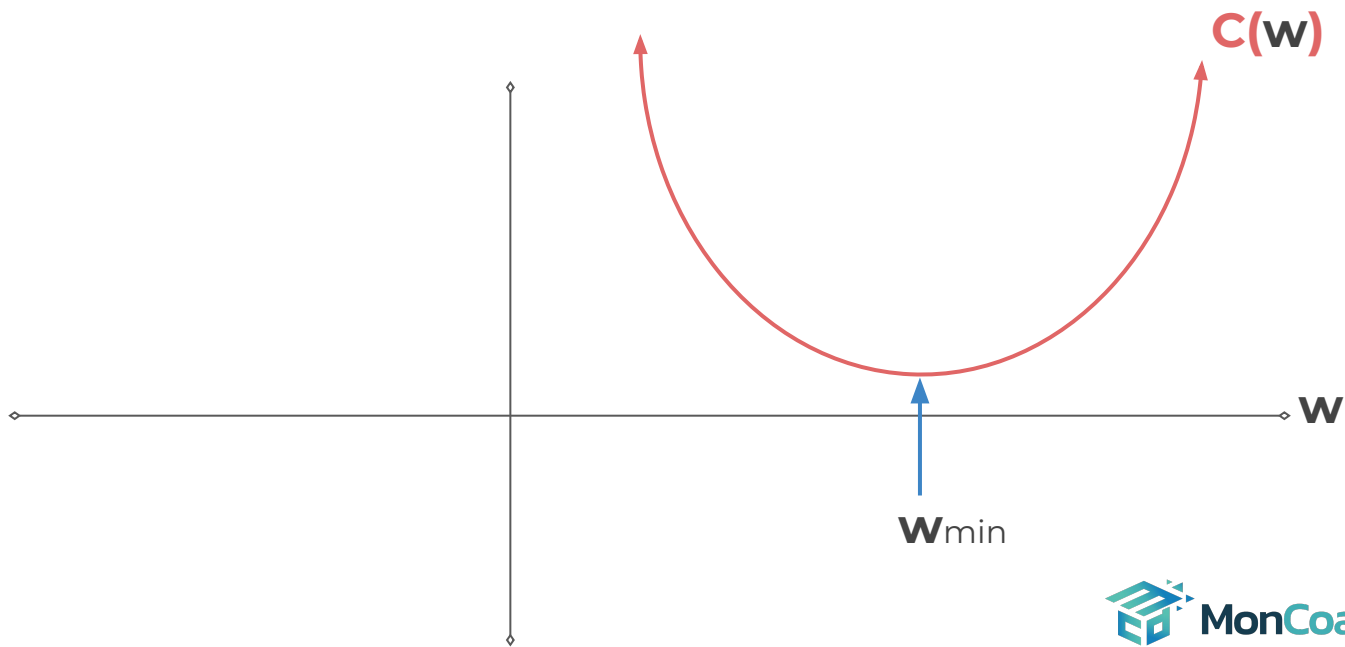
- Nous pouvons utiliser **la descente de gradient (gradient descent)** pour résoudre ce problème.





Deep Learning

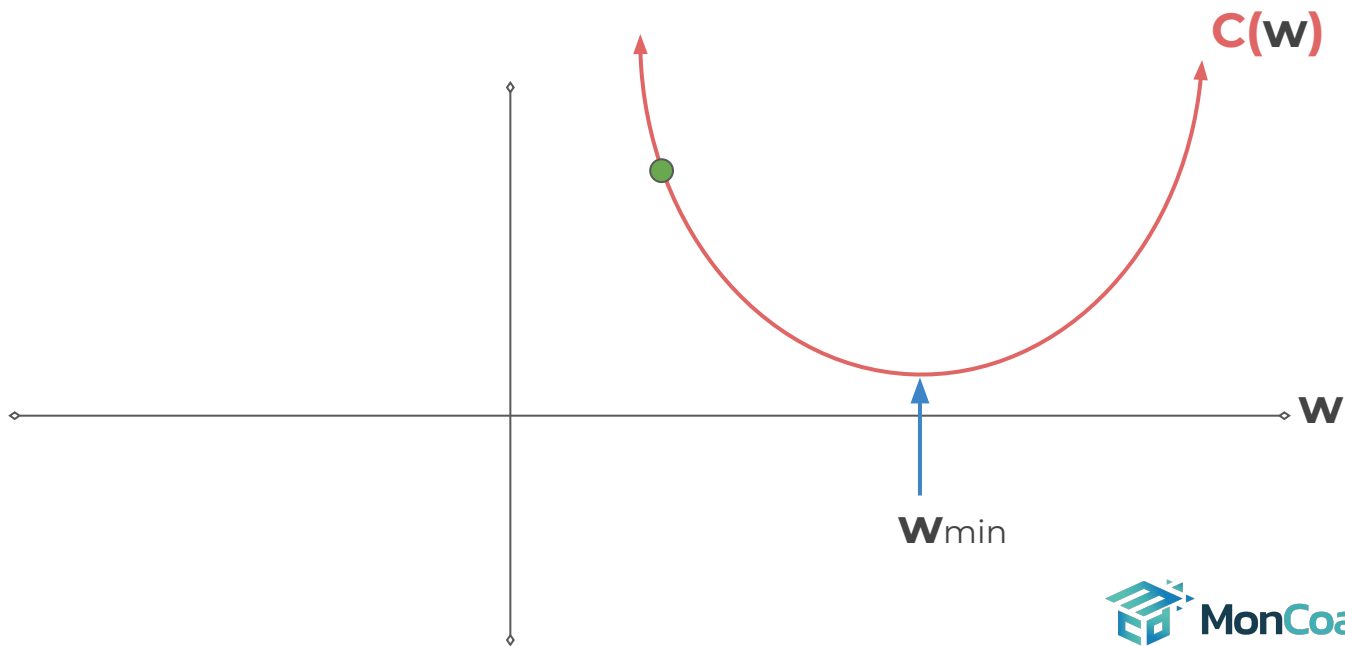
- Revenons à notre version simplifiée pour voir comment cela fonctionne.





Deep Learning

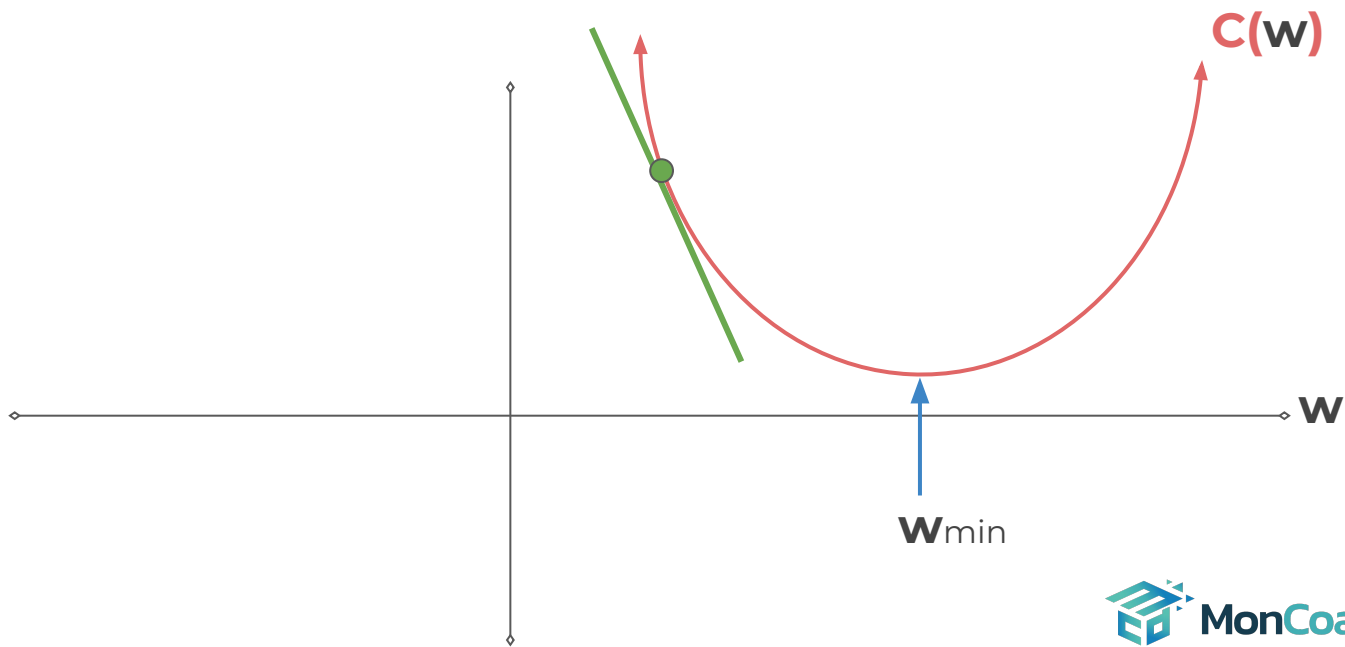
- On peut calculer la pente en un point





Deep Learning

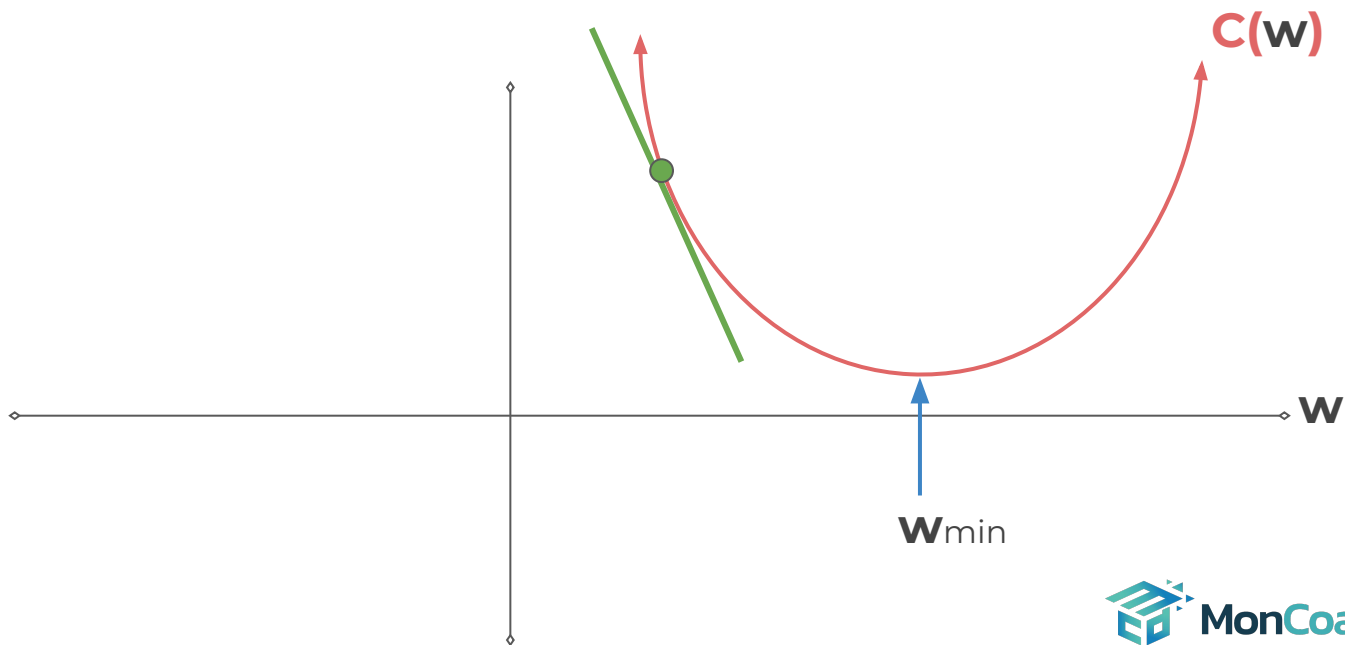
- On peut calculer la pente en un point





Deep Learning

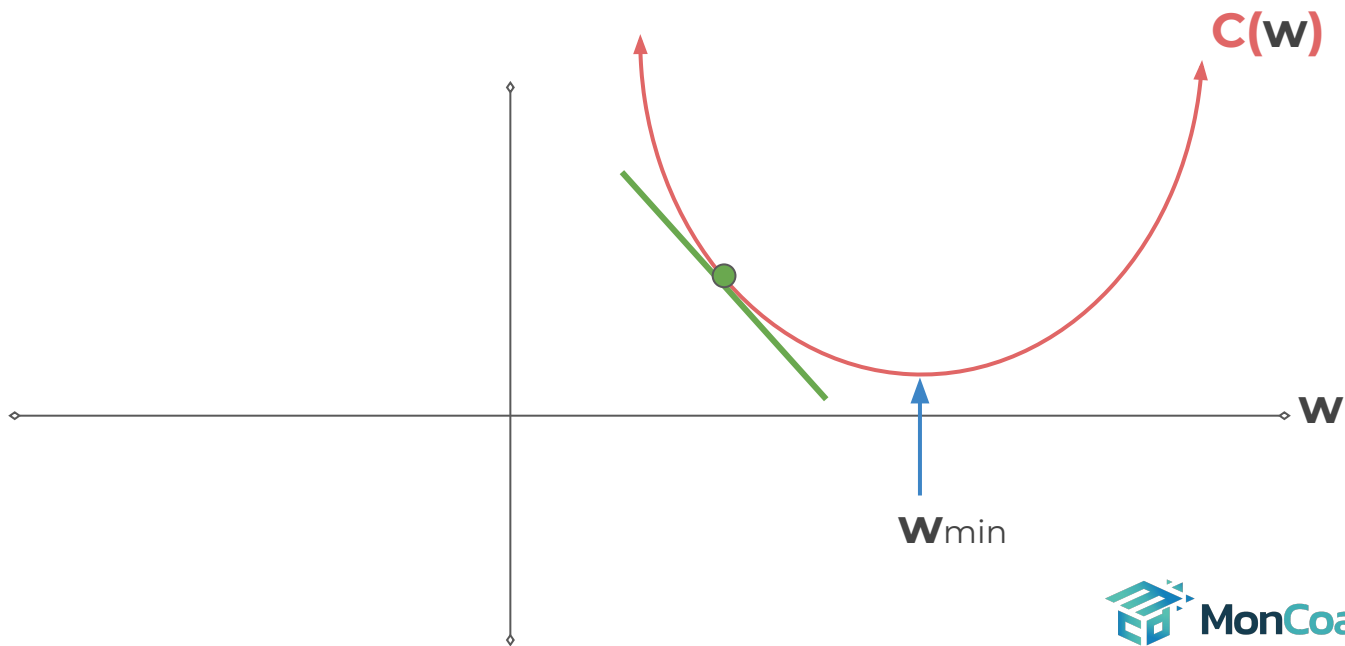
- Ensuite, nous nous déplaçons dans le sens descendant de la pente.





Deep Learning

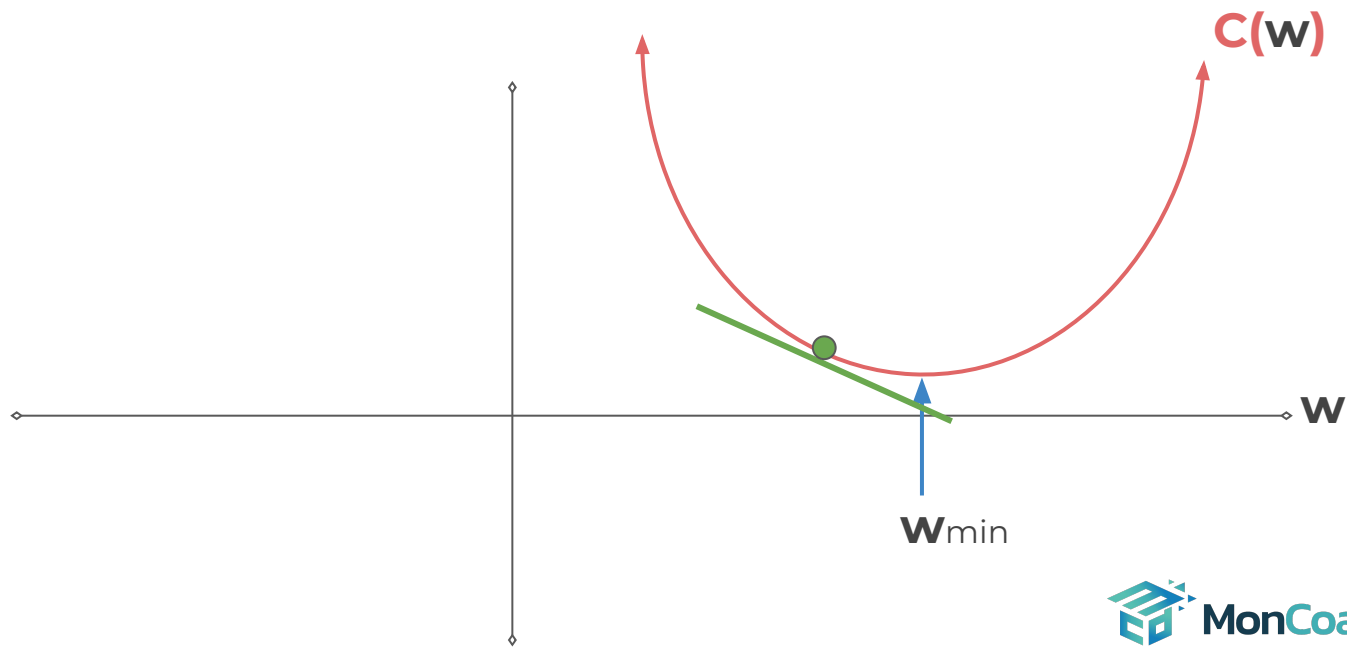
- Ensuite, nous nous déplaçons dans le sens descendant de la pente.





Deep Learning

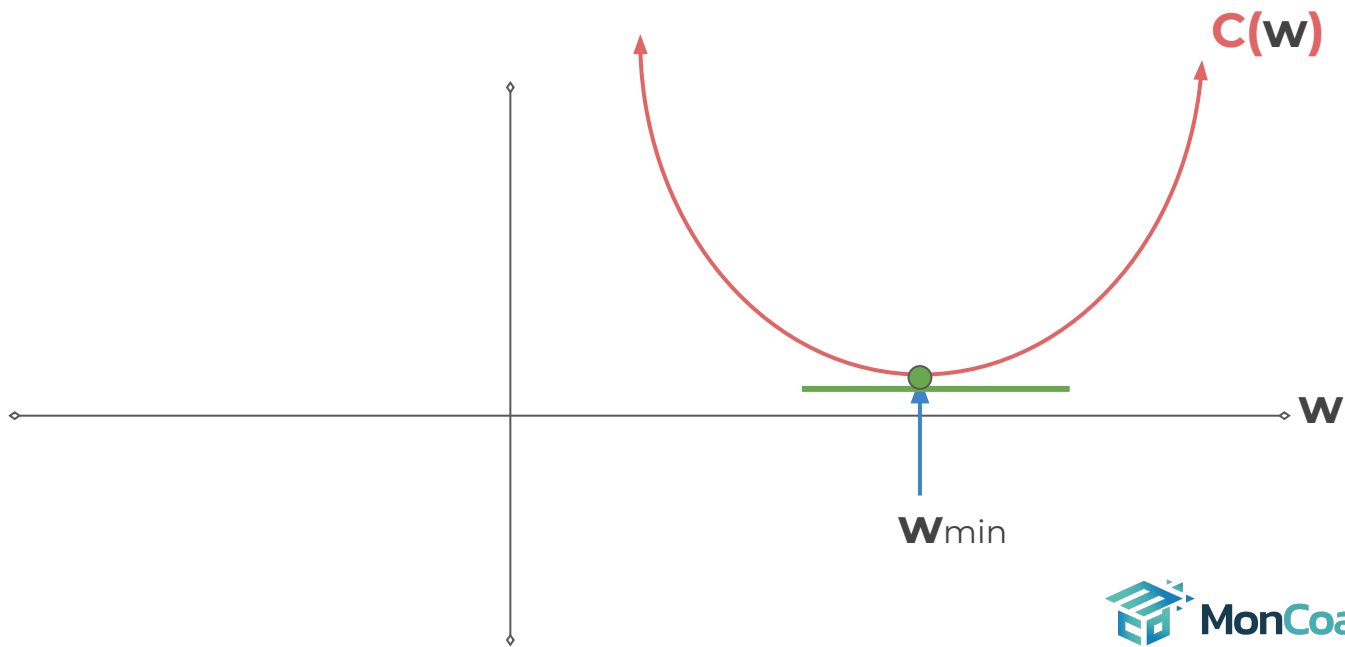
- Ensuite, nous nous déplaçons dans le sens descendant de la pente.





Deep Learning

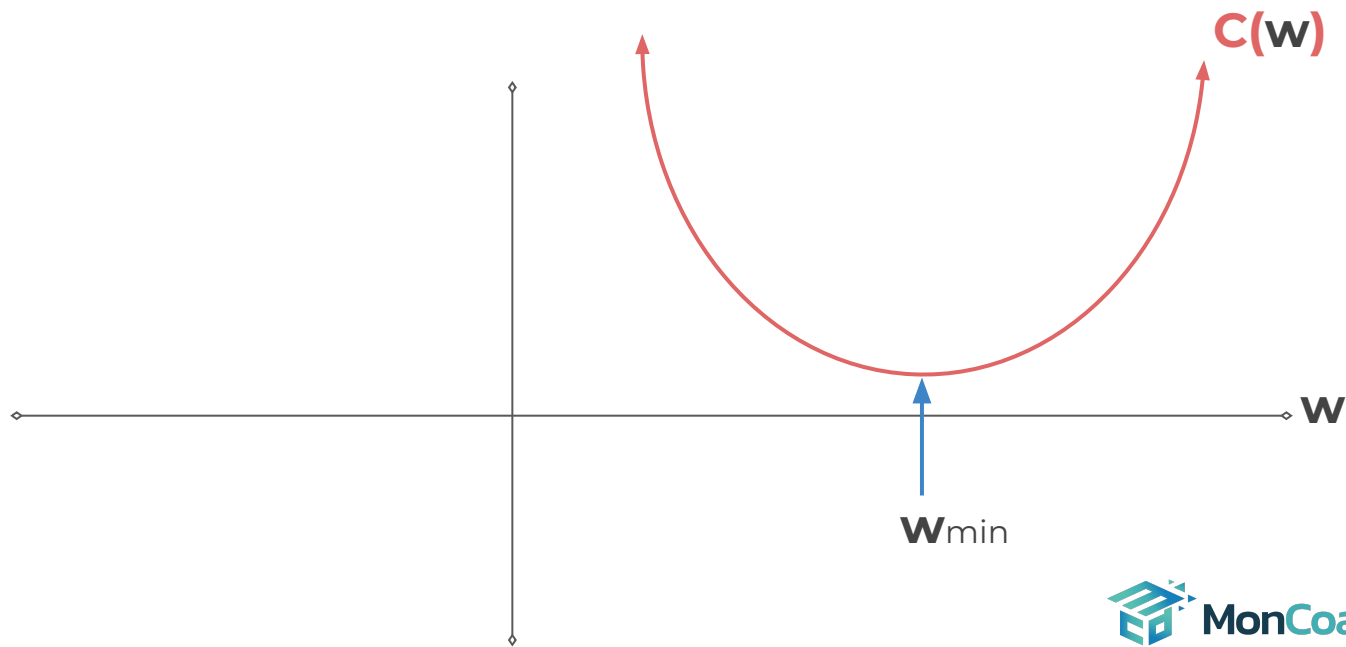
- Jusqu'à ce que nous convergions vers zéro, indiquant un minimum.





Deep Learning

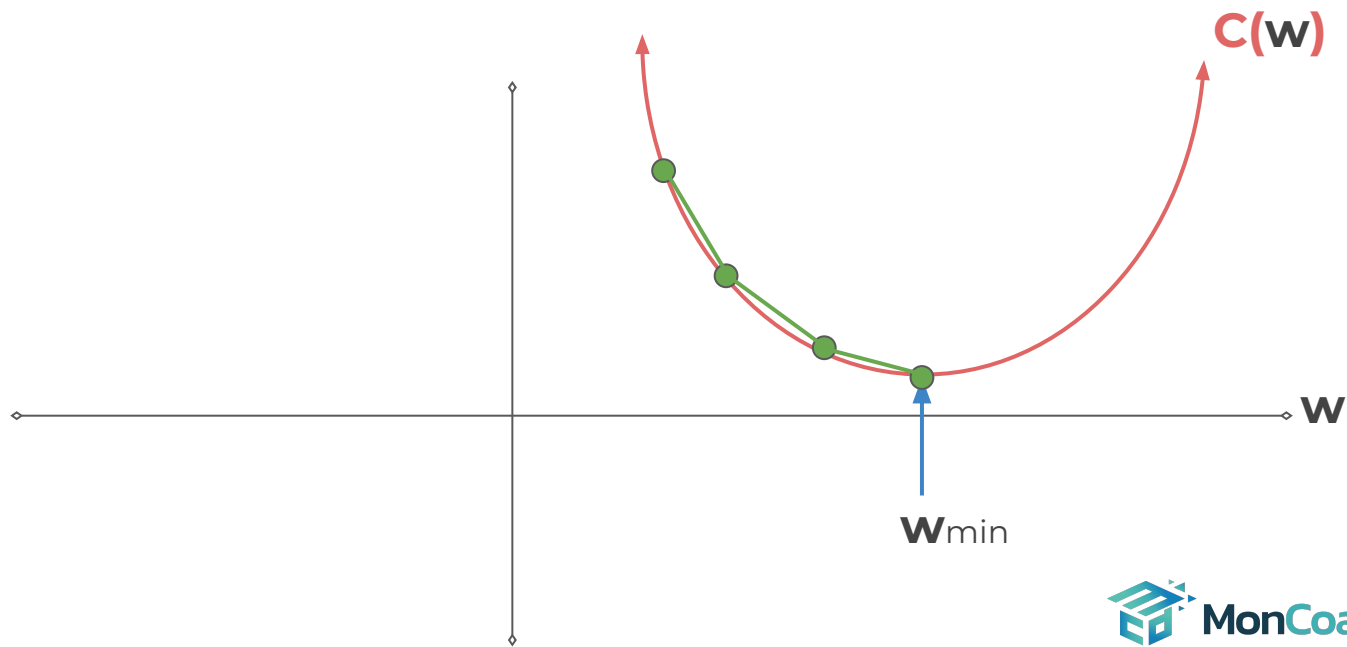
- Nous aurions pu changer la taille de notre “pas” pour trouver le point suivant !





Deep Learning

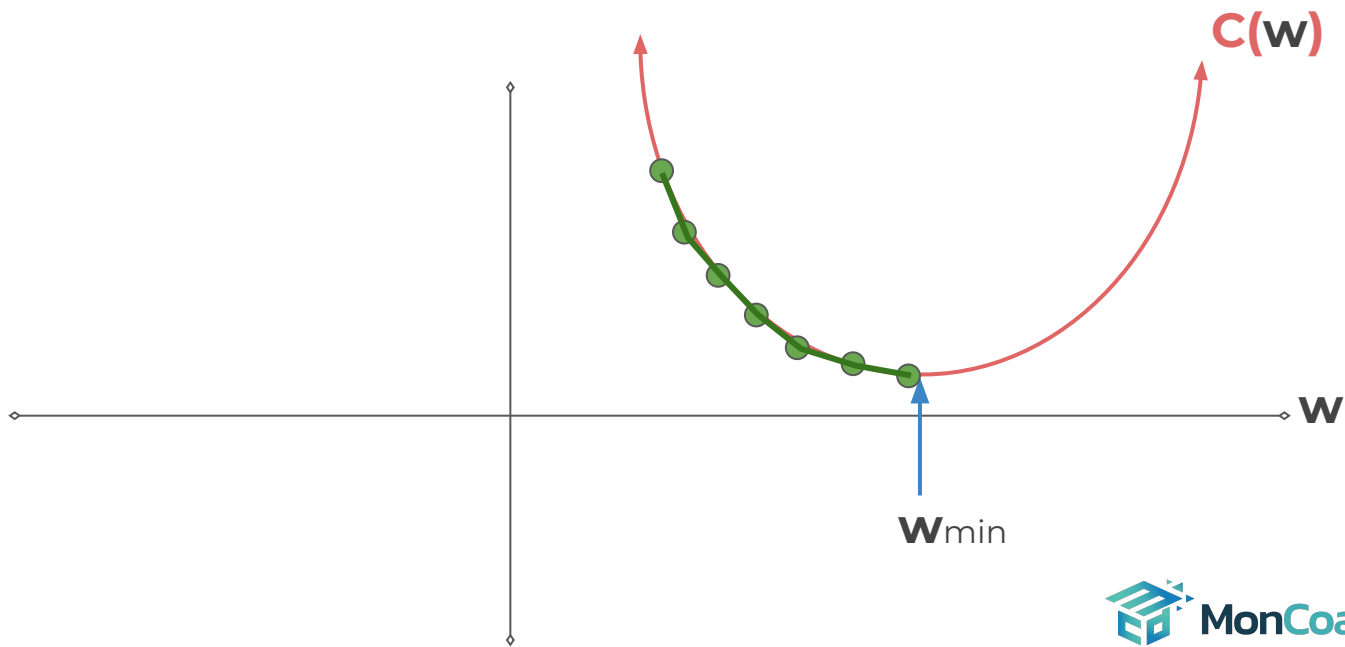
- Nos “pas” :





Deep Learning

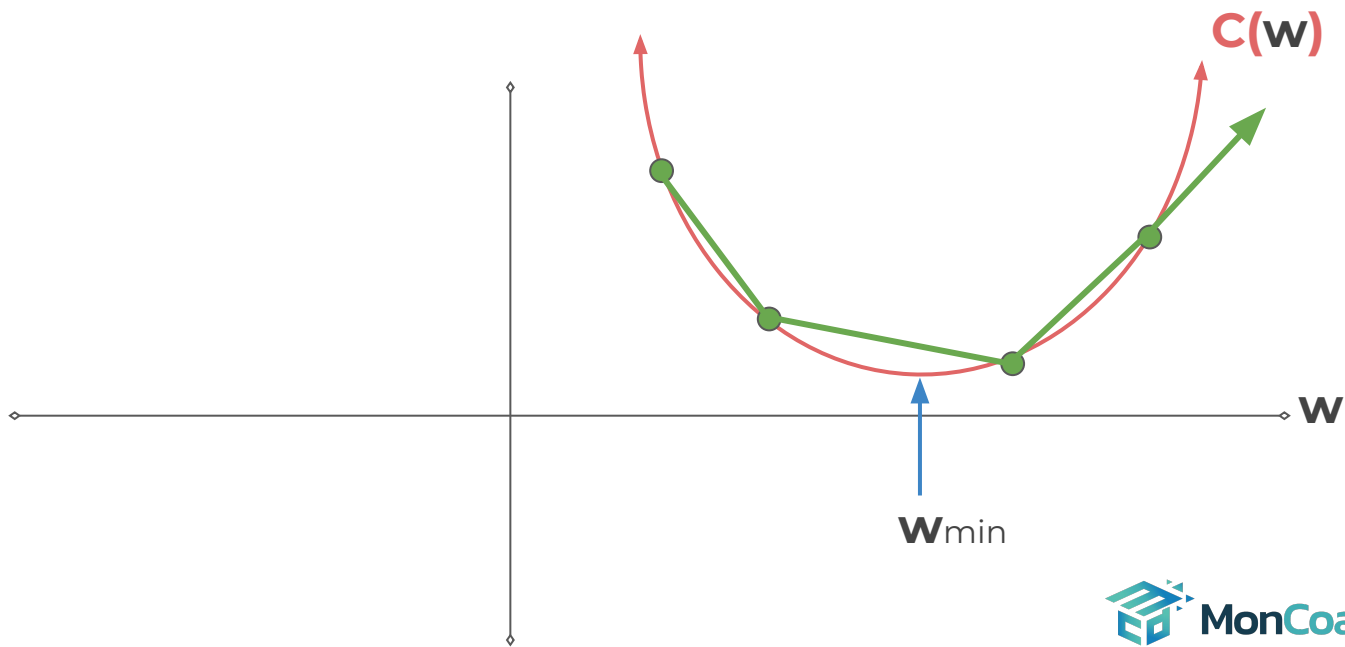
- Les petits pas prennent plus de temps pour trouver le minimum.





Deep Learning

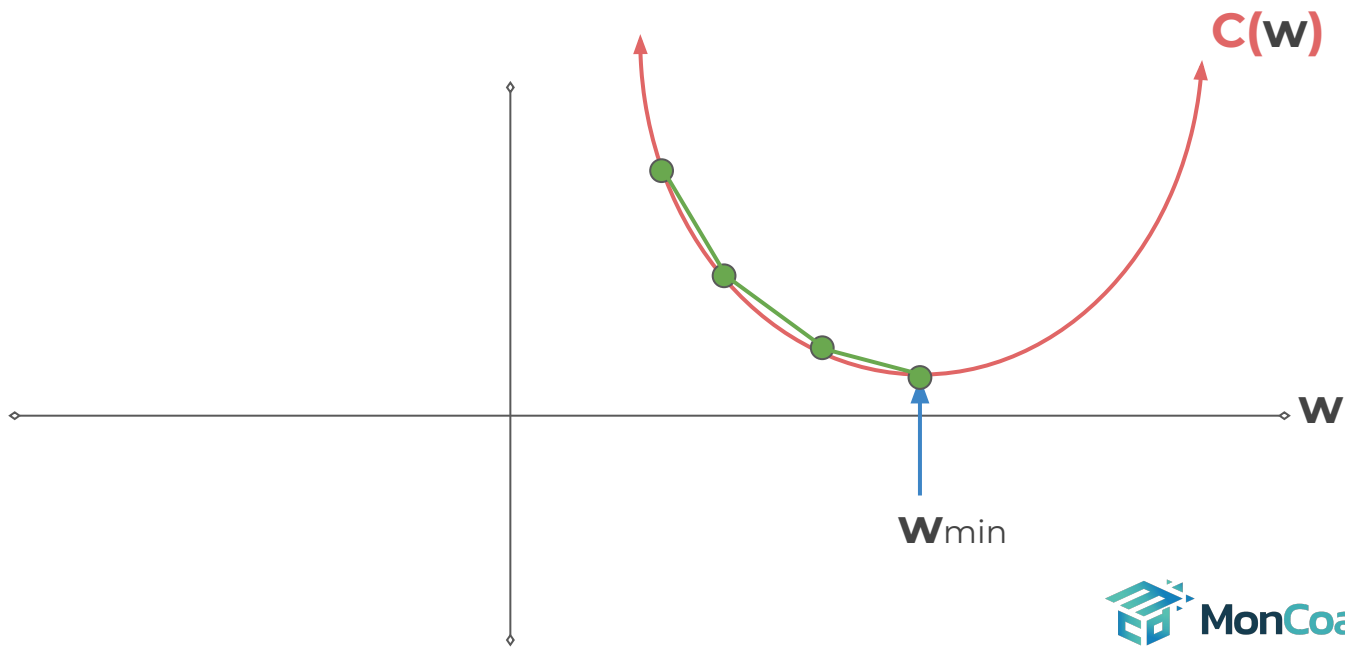
- Les plus grands pas sont plus rapides, mais nous risquons de dépasser le minimum !





Deep Learning

- Cette taille de pas est connue sous le nom de **taux d'apprentissage (learning rate)**.





Deep Learning

- Le taux d'apprentissage que nous avons montré dans nos illustrations était constant (chaque taille de pas était égale)
- Mais nous pouvons être plus malins et adapter notre taille de pas au fur et à mesure.





Deep Learning

- Nous pourrions commencer par de plus grands pas, puis aller plus petit à mesure que nous nous rendons compte que la pente se rapproche de zéro.
- C'est ce qu'on appelle **la descente adaptative de gradient (adaptive gradient descent)**.





Deep Learning

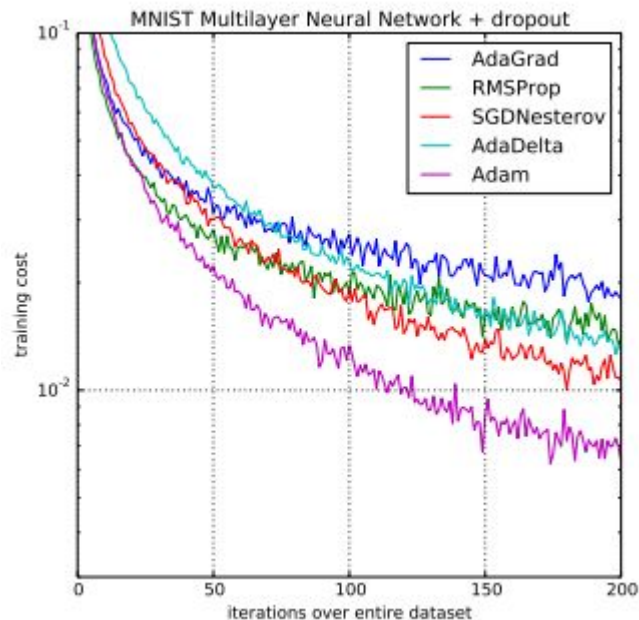
- En 2015, Kingma et Ba ont publié leur document : "Adam : une méthode d'optimisation stochastique".
- Adam est un moyen beaucoup plus efficace de rechercher ces minimums, vous nous verrez donc l'utiliser pour notre code !





Deep Learning

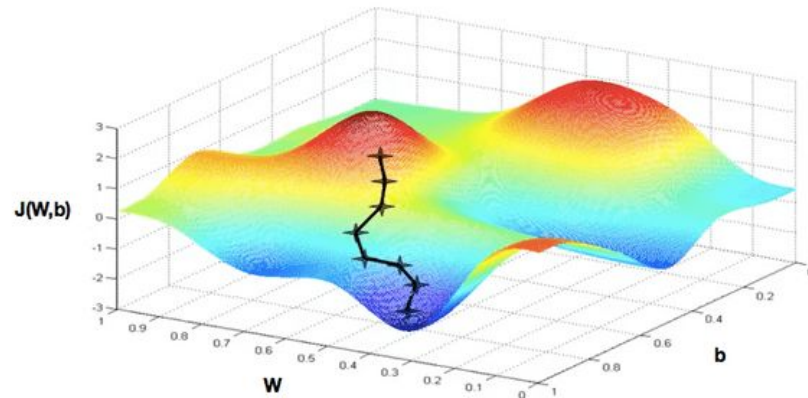
- Adam vs. d'autres algorithmes de descente de gradient :





Deep Learning

- De manière réaliste, nous calculons cette descente dans un espace n-dimensionnel pour tous nos poids.





Deep Learning

- Lorsqu'on traite ces vecteurs N-dimensionnels (tenseurs), la notation passe de la **dérivée** au **gradient**.
- Cela signifie que nous calculons $\nabla C(w_1, w_2, \dots, w_n)$





Deep Learning

- Pour les problèmes de classification, nous utilisons souvent la fonction de perte d'**entropie croisée**.
- L'hypothèse est que votre modèle prédit une distribution de probabilité $p(y=i)$ pour chaque classe $i=1,2,\dots,C$.





Deep Learning

- Pour une classification binaire, il en résulte :

$$-(y \log(p) + (1 - y) \log(1 - p))$$

- Pour **M** nombre de classes > 2

$$-\sum_{c=1}^M y_{o,c} \log(p_{o,c})$$





Deep Learning

- Récap :
 - Fonctions de Coût (Cost)
 - Descente de Gradient (Gradient Descent)
 - Optimiseur Adam
 - Coût quadratique et Entropie Croisée





Deep Learning

- Jusqu'à présent, nous comprenons comment les réseaux peuvent prendre en compte l'entrée, l'affecter avec des pondérations, des biais et des fonctions d'activation pour produire une prédiction en sortie.
- Nous avons ensuite appris comment évaluer ce résultat.





Deep Learning

- La dernière chose que nous devons apprendre sur la théorie est :
 - Une fois que nous avons obtenu la valeur de notre coût/perte, comment pouvons-nous réellement revenir en arrière et ajuster nos poids et nos biais ?
- C'est de la **rétropropagation (ou backpropagation)**, et c'est ce que nous allons couvrir dans la prochaine vidéo !



Backpropagation



MonCoachData





Deep Learning

- Le dernier sujet théorique que nous aborderons est la **rétropropagation (backpropagation)**.
- Nous commencerons par construire une intuition derrière la rétropropagation, puis nous nous plongerons dans le calcul et la notation de la rétropropagation.





Deep Learning

- Fondamentalement, nous voulons savoir comment les résultats de la fonction de coût changent par rapport aux pondérations dans le réseau, afin de pouvoir mettre à jour les pondérations pour minimiser la fonction de coût.





Deep Learning

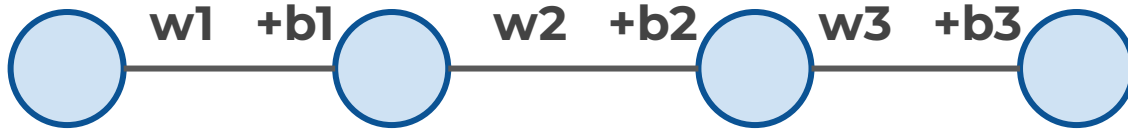
- Commençons par un réseau très simple, où chaque couche ne possède qu'un seul neurone





Deep Learning

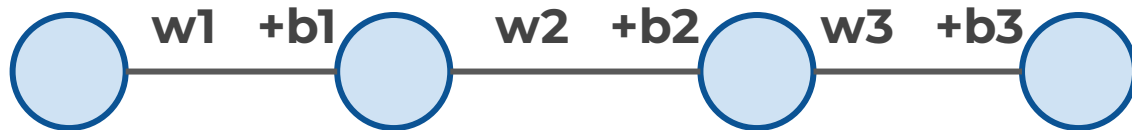
- Chaque entrée recevra un poids et un biais





Deep Learning

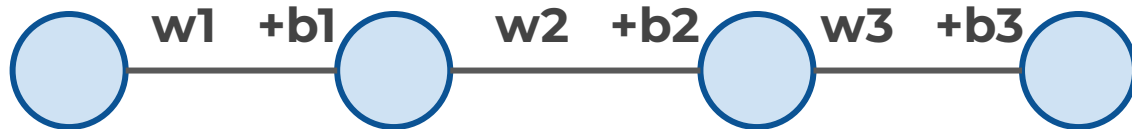
- Cela signifie que nous avons :
 - **$C(w_1, b_1, w_2, b_2, w_3, b_3)$**





Deep Learning

- Nous avons déjà vu comment ce processus se propage.
- Commençons par la fin pour voir la rétropropagation.





Deep Learning

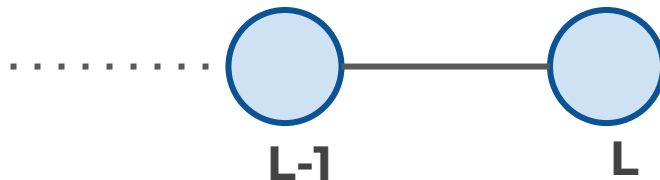
- Supposons que nous ayons des couches (layers) **L**, alors notre notation devient :





Deep Learning

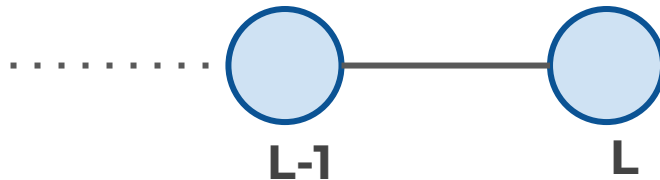
- En nous concentrant sur ces deux dernières couches, nous allons définir **$\mathbf{z} = \mathbf{w}\mathbf{x} + \mathbf{b}$**
- Ensuite, nous appliquerons une fonction d'activation : **$\mathbf{a} = \sigma(\mathbf{z})$**





Deep Learning

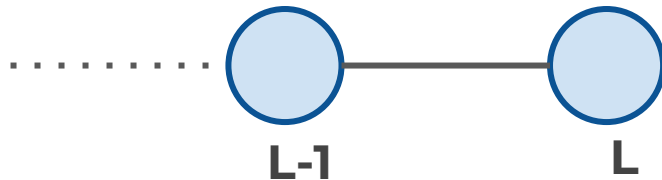
- Cela signifie que nous avons :
 - $\mathbf{z}^L = \mathbf{w}^L \mathbf{a}^{L-1} + \mathbf{b}^L$





Deep Learning

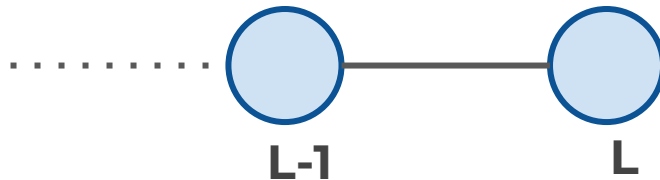
- Cela signifie que nous avons :
 - $\mathbf{z}^L = \mathbf{w}^L \mathbf{a}^{L-1} + \mathbf{b}^L$
 - $\mathbf{a}^L = \sigma(\mathbf{z}^L)$





Deep Learning

- Cela signifie que nous avons :
 - $\mathbf{z}^L = \mathbf{w}^L \mathbf{a}^{L-1} + \mathbf{b}^L$
 - $\mathbf{a}^L = \sigma(\mathbf{z}^L)$
 - $\mathbf{C}_0(\dots) = (\mathbf{a}^L - \mathbf{y})^2$

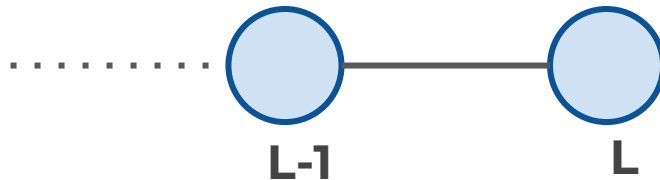




Deep Learning

- Nous voulons comprendre à quel point la fonction de coût est sensible aux variations de **w** :

$$\frac{\partial C_0}{\partial w^L}$$

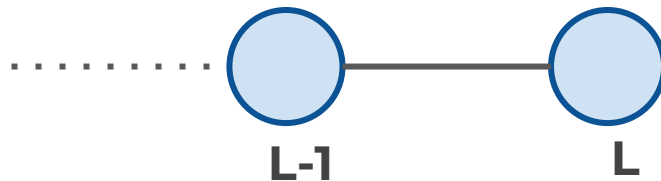




Deep Learning

- En utilisant les relations que nous connaissons déjà ainsi que la règle de dérivation en chaîne :

$$\frac{\partial C_0}{\partial w^L} = \frac{\partial z^L}{\partial w^L} \frac{\partial a^L}{\partial z^L} \frac{\partial C_0}{\partial a^L}$$

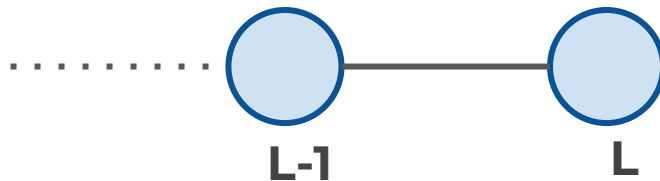




Deep Learning

- Nous pouvons faire le même calcul pour les termes de biais :

$$\frac{\partial C_0}{\partial b^L} = \frac{\partial z^L}{\partial b^L} \frac{\partial a^L}{\partial z^L} \frac{\partial C_0}{\partial a^L}$$





Deep Learning

- L'idée principale ici est que nous pouvons utiliser le gradient pour remonter dans le réseau et ajuster nos poids et biais afin de minimiser la sortie du vecteur d'erreur sur la dernière couche de sortie.





Deep Learning

- En utilisant une certaine notation de calcul, nous pouvons étendre cette idée aux réseaux comportant plusieurs neurones par couche.
- Produit Hadamard

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} \odot \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1 * 3 \\ 2 * 4 \end{bmatrix} = \begin{bmatrix} 3 \\ 8 \end{bmatrix}$$





Deep Learning

- Compte tenu de cette notation et de la rétropropagation, nous avons quelques étapes principales pour entraîner les réseaux de neurones.
- Note ! Vous n'avez pas besoin de comprendre entièrement ces détails complexes pour continuer avec les parties de code.





Deep Learning

- Étape 1 : En utilisant l'entrée **$\mathbf{x}$** , définissez la fonction d'activation **$\mathbf{a}$** pour la couche d'entrée.
 - **$\mathbf{z} = \mathbf{w} \mathbf{x} + \mathbf{b}$**
 - **$\mathbf{a} = \sigma(\mathbf{z})$**
- Le produit **$\mathbf{a}$** ainsi obtenu alimente la couche suivante (et ainsi de suite).





Deep Learning

- Étape 2 : Pour chaque couche, calculez :
 - $\mathbf{z}^L = \mathbf{w}^L \mathbf{a}^{L-1} + \mathbf{b}^L$
 - $\mathbf{a}^L = \sigma(\mathbf{z}^L)$





Deep Learning

- Étape 3 : Nous calculons notre vecteur d'erreur :
 - $\delta^L = \nabla_a C \odot \sigma'(z^L)$





Deep Learning

- Étape 3 : Nous calculons notre vecteur d'erreur :
 - $\delta^L = \nabla_a C \odot \sigma'(z^L)$
 - $\nabla_a C = (a^L - y)$
 - **Exprimer le taux de variation de C par rapport aux activations de sortie**



Deep Learning

- Étape 3 : Nous calculons notre vecteur d'erreur :
 - $\delta^L = (a^L - y) \odot \sigma'(z^L)$





Deep Learning

- Étape 3 : Nous calculons notre vecteur d'erreur :
 - $\delta^L = (a^L - y) \odot \sigma'(z^L)$
- Maintenant, écrivons notre terme d'erreur pour une couche en fonction de l'erreur de la couche suivante (puisque nous reculons dans le réseau).
- Note 1 : **l** minuscule pour la couche **L**
- Note 2 : Chiffre **1**





Deep Learning

- Étape 4 : Rétropropagation de l'erreur :
 - Pour chaque couche : $L-1, L-2, \dots$ nous calculons (notez la minuscule L (l)):
 - $\delta^l = (\mathbf{w}^{l+1})^T \delta^{l+1} \odot \sigma'(z^l)$
 - $(\mathbf{w}^{l+1})^T$ est la transposition de la matrice de poids de la couche $l+1$





Deep Learning

- Étape 4 : Rétropropagation de l'erreur :
 - Il s'agit de l'erreur généralisée pour toute couche l :
 - $\delta^l = (\mathbf{w}^{l+1})^T \delta^{l+1} \odot \sigma'(z^l)$
 - $(\mathbf{w}^{l+1})^T$ est la transposée de la matrice de poids de la couche $L+1$





Deep Learning

- Étape 4 : Lorsque nous appliquons la transposée de la matrice de poids $(\mathbf{w}^{l+1})^T$, on peut penser intuitivement que cela revient à déplacer l'erreur vers l'arrière dans le réseau, ce qui nous donne une sorte de mesure de l'erreur à la sortie de la l-ème couche.





Deep Learning

- Étape 4 : Nous prenons ensuite le produit Hadamard $\odot \sigma'(\mathbf{z}^l)$. Cela permet de déplacer l'erreur vers l'arrière à travers la fonction d'activation en couche l , nous donnant l'erreur δ^l dans l'entrée pondérée de la couche l .





Deep Learning

- Le gradient de la fonction de coût est donné par :
 - Pour chaque couche : $L-1, L-2, \dots$ nous calculons

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l$$

$$\frac{\partial C}{\partial b_j^l} = \delta_j^l$$





Deep Learning

- Cela nous permet ensuite d'ajuster les poids et les biais pour aider à minimiser cette fonction de coût.
- Consultez les liens externes pour plus de détails !



TensorFlow et Keras



MonCoachData





Deep Learning

- Avant de commencer à apprendre comment coder nos propres réseaux de neurones, clarifions rapidement les différences entre TensorFlow et Keras !





Deep Learning

- TensorFlow est une bibliothèque de Deep Learning en open-source développée par Google, TF 2.0 étant officiellement disponible depuis fin 2019.





Deep Learning

- TensorFlow possède un vaste écosystème de composants connexes, notamment des bibliothèques comme Tensorboard, des API de déploiement et de production, et la prise en charge de divers langages de programmation.





Deep Learning

- Keras est une bibliothèque python de haut niveau qui peut utiliser une variété de bibliothèques de Deep Learning en dessous, comme : TensorFlow, CNTK, ou Theano.





Deep Learning

- TensorFlow 1.x avait un système complexe de classes python pour la construction de modèles, et en raison de l'énorme popularité de Keras, lorsque TF 2.0 est sorti, TF a adopté Keras comme API officielle pour TF.





Deep Learning

- Bien que Keras reste une bibliothèque séparée de Tensorflow, elle peut maintenant être officiellement importée par TF, il n'est donc plus nécessaire de l'installer en plus.





Deep Learning

- L'API de Keras est facile à utiliser et construit des modèles en ajoutant simplement des couches les unes sur les autres par de simples appels.
- Voyons maintenant les bases de l'API de Keras pour TensorFlow !



Les bases de la syntaxe Keras



MonCoachData

Les bases de la syntaxe Keras

PARTIE 2



MonCoachData



Les bases de la syntaxe Keras

PARTIE 3



MonCoachData



Régression avec Keras

PARTIE 2 - CODE



MonCoachData



Régression avec Keras

PARTIE 2 - CODE



MonCoachData



Régression avec Keras

PARTIE 3 - CODE



MonCoachData



Classification avec Keras

PARTIE 1 - CODE



MonCoachData





Deep Learning

- Cette vidéo montrera comment effectuer une tâche de classification avec TensorFlow.
- Nous nous concentrerons également sur la manière d'identifier et de traiter l'overfitting par le biais de Callbacks Early Stop et des couches de Dropout.





Deep Learning

- Arrêt anticipé - Early Stop
 - Keras peut arrêter automatiquement l'entraînement en fonction d'une condition de perte sur les données de validation passées lors de l'appel `model.fit()`.





Deep Learning

- Dropout Layers
 - Un Dropout peut être ajouté aux couches pour "éteindre" des neurones pendant l'entraînement afin d'éviter le surapprentissage (overfitting).





Deep Learning

- Dropout Layers
 - Chaque couche Dropout “diminue” d'un pourcentage d'unités de neurone défini par l'utilisateur dans la couche précédente, à chaque batch (lot).



Classification avec Keras

PARTIE 2 : CODE



MonCoachData



Introduction au Projet



MonCoachData





Deep Learning

- C'est le moment d'attaquer le projet !
Vous allez construire un modèle qui tentera de prédire si quelqu'un remboursera ou non son prêt en se basant sur des informations historiques.





Deep Learning

- Vous avez 3 options pour ce projet
 - Entièrement en solo
 - En suivant le guide du notebook Exercices
 - Descendre le code avec les vidéos de correction





Deep Learning

- Entièrement en solo
 - Lisez l'introduction dans le notebook d'exercices, puis appliquez vos propres méthodes pour construire un modèle prédictif.





Deep Learning

- Guide du Notebook Exercices
 - Suivez les étapes écrites dans le notebook pour accomplir les tâches qui vont vous guider dans la construction d'un modèle prédictif.





Deep Learning

- Code des vidéos Solution
 - Sauter la prochaine vidéo et suivez avec nous pour vous guider à travers notre vidéo de solution pour le projet.





Deep Learning

- Il s'agit d'un gros projet !
- Pour refléter une situation réaliste, nous passerons beaucoup de temps à réaliser du feature engineering et à analyser nos données.
- Découvrons le notebook du projet lors de la prochaine vidéo !



Projet Keras

Aperçu de l'Exercice



Projet Keras Solutions Exercice

ANALYSE EXPLORATOIRE DES DONNÉES



MonCoachData



Projet Keras Solutions Exercice

PRÉ-TRAITEMENT DES DONNÉES : DONNÉES
MANQUANTES



MonCoachData



Projet Keras Solutions Exercice

DONNÉES CATÉGORIELLES



MonCoachData



Projet Keras Solutions Exercice

PRÉ-TRAITEMENT DES DONNÉES



MonCoachData



Projet Keras Solutions Exercice

CRÉATION ET ENTRAÎNEMENT DU MODÈLE



MonCoachData



Projet Keras Solutions Exercice

ÉVALUATION DU MODÈLE



MonCoachData



Tensorboard





Deep Learning

- Tensorboard est un outil de visualisation de Google conçu pour fonctionner en parallèle de TensorFlow afin de visualiser les différents aspects de votre modèle.





Deep Learning

- Ici, nous allons simplement comprendre comment visualiser le tableau de bord Tensorboard et analyser un modèle existant.
- **REMARQUE - Si vous ne codez pas sur un notebook, cela nécessite que vous compreniez les chemins d'accès des fichiers et l'emplacement de votre notebook ou de votre fichier .py ! Lisez bien le notebook tensorboard du cours dans ce cas.**





Deep Learning

- N'oubliez pas que Tensorboard est une bibliothèque distincte de TensorFlow.
- Les utilisateurs de Google Collab peuvent suivre aussi le guide officiel de Google et le Notebook suivant :

https://www.tensorflow.org/tensorboard/tensorboard_in_notebooks

